GH DEVELOPER LEARNING SERIES

2026 EDITION · PDF

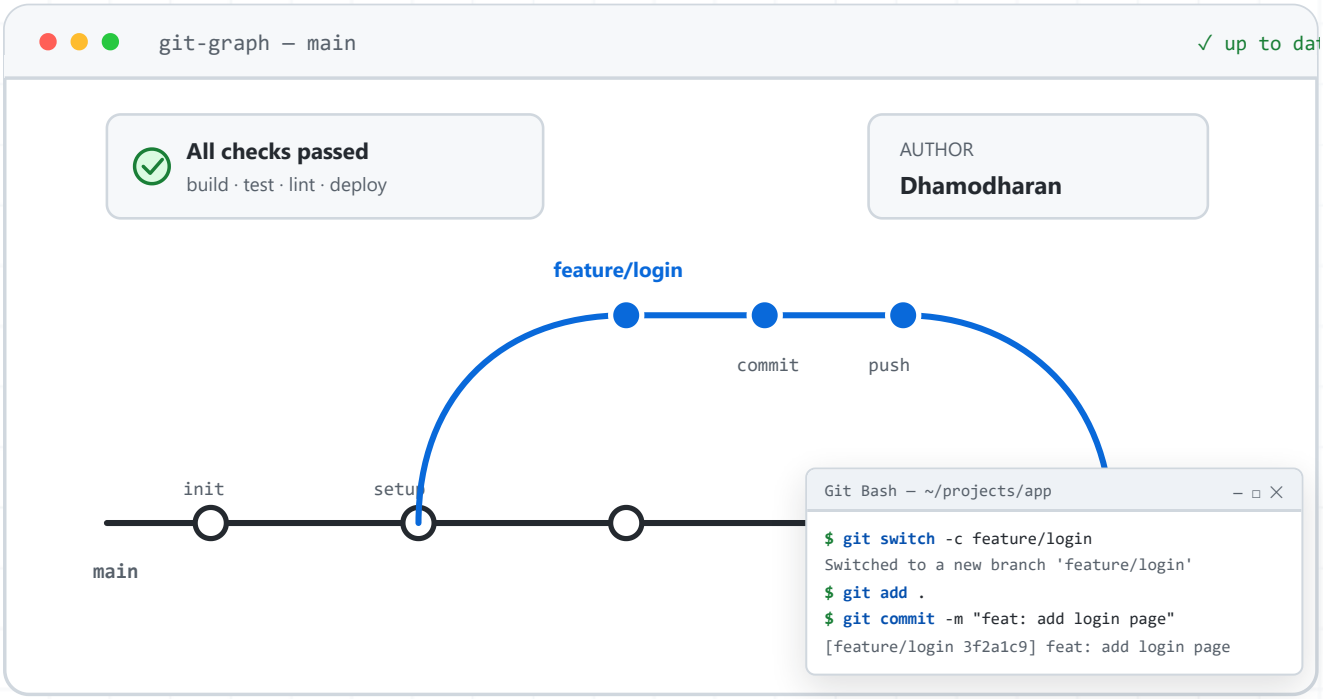
PRACTICAL · REAL-TIME · INTERVIEW-READY

GIT & GITHUB

COMPLETE DEVELOPER GUIDE

Beginner to Advanced | Practical Learning | Real-Time IT Company Workflow

13 Chapters | 150+ Topics | 100+ Commands | 22 Visual Walkthroughs | Interview Q&A



WRITTEN & ILLUSTRATED BY

Dhamodharan

Git 2.x · GitHub Actions · Pages · Security

For Students, Java Full Stack & React Developers

Simple Tanglish explanations · A4 Print Edition

START HERE

How to Use This Book

Indha book ungalai zero-la irundhu **real IT company developer** level-ku kondu varum. Git basics-la start panni, GitHub collaboration, Actions, Security, Pages, and real team workflow varaikkum step by step cover pannirukkom. Every chapter-la explanation, command, expected output, screenshot-style visual, and real-time usage irukku.

- **Who is this book for?**

Students & freshers who want their first developer job. **Java Full Stack** and **React developers** who need Git for daily work. **Job seekers** preparing for Git interview questions.

- **What should you know already?**

Basic computer usage and one programming language is enough. Git experience **not required**. Windows commands ellam Git Bash-la irundhu explain pannirukkom.

1. How every topic is explained

What & Why
Concept enna, endhukku use pandrom — simple Tanglish-la.

Syntax & Example
Exact command, real file names, and ready-to-run example.

Output & Usage
Expected terminal output plus IT company real-time usage.

Important topics-la **step-by-step numbered instructions**, **comparison tables**, and **common mistakes** add pannirukkom. Chapter end-la quick recap and practice task irukku.

2. Visual legend

ILLUSTRATIVE MOCKUP

UI walkthrough visuals. Indha book-la irukkura GitHub, VS Code, Git Bash, and Windows screens ellame **clearly labelled illustrative mockups** — actual software-la irundhu edutha screenshots illa. Steps follow pannina ungal screen-la idhe maadhiri UI varum. Ellaa commands, outputs, and workflows real and runnable.

terminal
```
$ git status
```

VS Code Light+ code editor. File tab, line numbers, syntax colours. Command copy panni terminal-la paste pannalaam. Dark code blocks use pannala.

Git Bash output
```
On branch main
nothing to commit
```

Terminal output. Neenga expect pannavendiya result. Output text machine-la irundhu copy pannina maadhiri same-a irukkum.

Best practice
Warning

Callouts. Green = follow pannanum. Amber = mistake pannadheenga. Blue = concept note. Red = data loss risk.

BEFORE YOU START

Git install pannunga, GitHub account create pannunga, and VS Code install pannunga. Chapter 1-la installation steps full-a irukku. Practice panna oru empty folder create pannikonga — `C:\projects\git-practice`. Book-la ellaa examples-um indha folder-la irundhu run pannalaam.

> *"Git theriyadha developer-ah today IT company-la work panna mudiyadhu. Idhu ungal daily tool — indha book adhai practical-a kathukkum."*

CONTENTS

What's inside

13 chapters · 150+ topics · from your first `git init` to a complete company release workflow.

01 Git Fundamentals 4
Version control, Git vs GitHub, architecture, installation, configuration, first repository
Version control · Git vs SVN · 3 areas · Install · git config · git init

02 Basic Git Commands 9
The 13 commands you will type every single working day
status · add · commit · log · diff · .gitignore · restore · clean

03 GitHub Repository Management 14
Accounts, repositories, README, visibility, settings, releases
Create repo · Public vs private · README · License · Settings · Releases

04 Remote Repository 18
Connect local code to GitHub and keep both sides in sync
remote add · push · pull · fetch · clone · SSH vs HTTPS

05 Branching and Merging 23
Parallel development, merge types, conflicts, rebase, Git Flow
switch -c · fast-forward · 3-way merge · conflicts · rebase · cherry-pick

06 Advanced Git 28
Stash, reset, revert, reflog, tags, hooks, worktrees, bisect
stash · reset · revert · reflog · interactive rebase · hooks · LFS

07 GitHub Collaboration 35
Fork, Pull Requests, reviews, CODEOWNERS, protected branches, Issues
Fork · Pull Request · Code review · Merge strategies · CODEOWNERS · Issues

Part 1 · Foundations
Chapters 1–4. Git mental model, daily commands, GitHub repos, and remotes.

Part 2 · Team skills
Chapters 5–7. Branching, advanced recovery, PR review, and collaboration.

Part 3 · Professional
Chapters 8–13. CI/CD, security, projects, Pages, and real company workflow.

BEFORE YOU BEGIN

INSTALL THESE FOUR
Git for Windows (includes Git Bash) · **VS Code** · **Node.js LTS** (React examples) · **JDK 17 + Maven** (Spring Boot examples). GitHub account free plan is enough.

HOW TO PRACTISE
Oru `git-practice` folder create pannunga. Every chapter-la irukkura commands adhula run pannunga. Command output match aagalana error message-ah Appendix B-la search pannunga.

CONTENTS · CONTINUED

Chapters 8 to 13 & appendices

08 **GitHub Actions & CI/CD** 41
Automate build, test, and deploy on every push and Pull Request
Workflow YAML · Triggers · Jobs · Runners · Matrix · Artifacts

09 **GitHub Security** 46
Protect your account, code, and secrets like a real company
2FA · SSH keys · PAT · Secrets · Dependabot · CodeQL

10 **GitHub Project Management** 50
Issues, labels, milestones, Projects boards, and roadmaps
Issues · Labels · Milestones · Projects · Views · Roadmap

11 **GitHub Pages & Deployment** 53
Publish a live website directly from your repository
Pages setup · Actions deploy · Custom domain · HTTPS · React SPA

12 **Professional GitHub Skills** 56
Profile README, open source, commit conventions, versioning
Profile · Open source · Semantic commits · SemVer · GitHub CLI

13 **Real-Time IT Company Workflow** 60
A complete React + Spring Boot team project, sprint by sprint
Team roles · Branch naming · PR flow · Reviews · Conflicts · Deploy

APPENDICES & PRACTICE

A **Git Commands Cheat Sheet** 65
Every command from the book on two pages

B **Common Git Errors and Solutions** 67
Real error messages with fixes

C **Git Interview Questions & Answers** 68
12 questions asked in fresher and developer interviews

D **Practice Tasks & Capstone Project** 69
Ten graded tasks and a seven-day plan

E **Conclusion & Next Steps** 70
Your 30-day plan after finishing this book

STUDY PLAN

Week 1: Chapters 1–3 + daily command drills. **Week 2:** Chapters 4–6 + branch and conflict practice. **Week 3:** Chapters 7–9 + one Pull Request per day. **Week 4:** Chapters 10–13 + practice tasks from Appendix D.

CHAPTER 01 · FOUNDATIONS

Git Fundamentals

Version control enna, Git eppadi work aagum, and ungal computer-la first repository create pannuvom.

9 topics | Level: **Beginner** | Time: **45 min** | Tool: **Git Bash + VS Code**

Oru company-la 10 developers same project-la work pandranga. Oruvar code edit pannumbodhu innoruvar overwrite pannita enna aagum? Bug fix panna previous version-ku thirumba poganum-na eppadi? Indha problems ellam solve panna dhaan **Version Control System (VCS)** use pandrom. Git ippo world-la most used VCS — 90%+ professional teams idhai dhaan use pandranga.

1.1 What is Version Control?

DEFINITION

Version Control System na oru software tool. Idhu files-la nadakkura **every change-ah track** pannum, **who changed what and when** nu record pannum, and needed-aana **any previous version-ku return** poga help pannum.

Simple example: ungal resume file `resume_v1.docx`, `resume_v2_final.docx`, `resume_final_final.docx` nu save pandringa. Idhu manual version control. Problem: edhu latest nu confusion, old version compare panna mudiyadhu, team-la share panna mudiyadhu. Git indha ellathayum automatic-a manage pannum.

Track
Every file change, author, timestamp, and reason (commit message) record aagum.

Revert
Bug vandha yesterday version-ku or single file-ah thirumba edukkalaam.

Collaborate
Multiple developers same repo-la branch use panni safe-a work pannalaam.

1.2 Why Git? (Features)

- **Distributed:** full history ungal laptop-la irukkum; internet illaama kooda commit pannalaam.
- **Fast:** almost ellaa operations local-a run aagum — milliseconds-la.
- **Branching:** branch create panna cheap and instant; merge easy.
- **Data integrity:** every commit-ku SHA-1 hash; file corrupt aana Git kandupidikkum.
- **Open source:** free, huge community, all OS support.
- **Industry standard:** GitHub, GitLab, Bitbucket ellame Git use pandranga.

1.3 Git vs GitHub

Freshers most common confusion idhu dhaan. **Git is a tool** (ungal computer-la run aagum). **GitHub is a website/service** (Git repositories cloud-la store panna). Git illaama GitHub use panna mudiyadhu; GitHub illaama Git use panna mudiyum.

Point	Git	GitHub
What	Version control software	Cloud hosting + collaboration platform
Where it runs	Local laptop / server	Browser + cloud servers
Created by	Linus Torvalds (2005)	GitHub Inc. (2008), now Microsoft
Internet needed?	No (for local work)	Yes
Main use	Track code versions	Store repos, Pull Requests, Actions, Issues
Alternatives	SVN, Mercurial	GitLab, Bitbucket, Azure DevOps

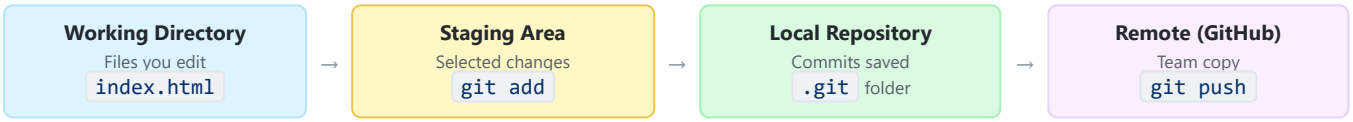
1.4 Git vs SVN

Company interviews-la ketpaanga: "Why Git, why not SVN?" Answer simple — SVN is a **centralized** system, Git is **distributed**. SVN-la server down aana commit panna mudiyadhu. Git-la local history irukku, so work continue pannalaam.

Feature	Git (Distributed)	SVN (Centralized)
Repository model	Every developer-ku full repo copy	One central server repo
Commit speed	Local, very fast	Network round-trip needed
Offline work	Full support	Very limited
Branching	Lightweight, instant	Heavy directory copy
Merging	Powerful 3-way merge + rebase	Basic, conflict-prone
History	Distributed; can't be lost easily	Central server only
Learning curve	Steeper (staging area)	Simpler concept

1.5 Git Architecture — The Three Areas

Git-la work pandra idam moonu. Indha mental model clear-a irundha 90% confusion poidum. **Working Directory**-la edit pandringa, **Staging Area**-la commit-ku select pandringa, **Local Repository**-la permanent-a save pandringa.



Area	Command in	What happens there
Working Directory	`edit files`	VS Code-la code write / modify pandringa. Git still track pannala.
Staging Area (index)	`git add`	Next commit-la enna files poganum nu select pandringa.
Local Repository	`git commit`	Snapshot permanent-a `.git` folder-la save aagum.
Remote Repository	`git push`	GitHub server-ku commits upload aagum; team share pandranga.

MENTAL MODEL

Online shopping maadhiri: **Working Directory** = shop-la items paakardhu. **Staging Area** = cart-la add pandradhu. **Commit** = order place pandradhu. **Push** = delivery address (GitHub)-ku anuppuradhu. Cart-la add pannaama order place panna mudiyaadhu — adhe maadhiri `git add` illaama commit panna mudiyaadhu (new files-ku).

1.6 File Lifecycle — Untracked to Committed

Oru file Git-la four states irukkum. `git status` command exactly indha state-ah dhaan kaattum.

State	Meaning	git status says	Next action
Untracked	Git inime paakala; new file	`Untracked files:`	`git add`
Modified	Tracked file changed, not staged	`Changes not staged`	`git add`
Staged	Ready for next commit	`Changes to be committed`	`git commit`
Committed	Saved in local history	`nothing to commit`	`git push`

Note: `.gitignore`-la list panna files (node_modules, .env, target/) Git ignore pannum — avai untracked-a kooda kaattadhu. Idhu Chapter 2-la detail-a paakkom.

1.7 Install Git on Windows

Official site **git-scm.com/download/win**-la irundhu installer download pannunga. Installer-la default options-ah continue pannalaam; but indha three choices mattum note pannunga.

1. **Download & run**
   `Git-2.x-64-bit.exe` download panni double-click pannunga. User Account Control prompt-la **Yes** click pannunga.
2. **Default editor**
   Use Visual Studio Code as Git's default editor select pannunga. Illa-na Notepad + Nano kooda work aagum.
3. **PATH option**
   Git from the command line and also from 3rd-party software (recommended) select pannunga — appo `git` command PowerShell/CMD-la kooda work aagum.
4. **Line endings**
   Checkout Windows-style, commit Unix-style line endings select pannunga. Windows + Linux team mix-la idhu safest.
5. **Finish & verify**
   Install click panni, Start menu-la **Git Bash** open pannunga, `git --version` type pannunga.

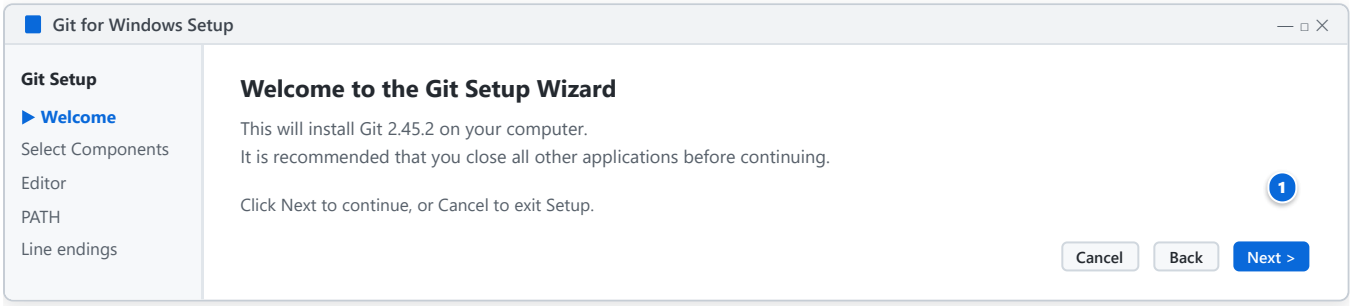


Fig 1.1 ILLUSTRATIVE MOCKUP **Git for Windows installer.** Welcome screen — Next click panni default options-la install pannalaam. Step 3-la PATH option mattum change pannunga.

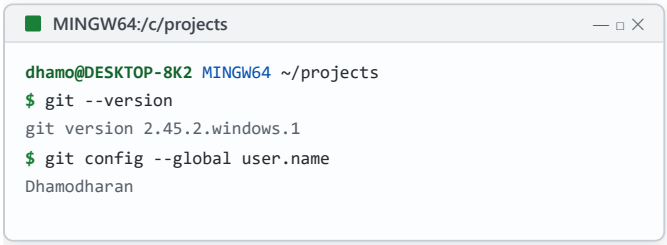


Fig 1.2 ILLUSTRATIVE MOCKUP **Git Bash after install.** Version output vandha install success.

1.7.1 Which terminal to use?

Git Bash — recommended for beginners; Linux-style commands (`ls`, `pwd`, `cat`) work aagum. **PowerShell / CMD** — work aagum, but some examples-la path syntax differ aagum. **VS Code terminal** — `Ctrl + ` ` open pannunga, terminal dropdown-la Git Bash select pannunga. Ellaa chapters-lum Git Bash examples use pandrom.

VERIFY CHECKLIST

- `git --version` → version number
- `git config --global user.name` → your name
- `git help -a` → command list

1.7.2 Install on macOS & Linux (short note)

macOS: `brew install git` (Homebrew) or `xcode-select --install`. **Ubuntu/Debian:** `sudo apt update && sudo apt install git`. **Fedora:** `sudo dnf install git`. Verify with `git --version`.

1.8 Configure Git (One-Time Setup)

Install pannina odane idhu pannunga. Indha details every commit-la author-ah save aagum. Wrong email vitta GitHub contributions ungal profile-la kaattadhu.

git-setup.sh

```
# 1. Identity — every commit-la idhu dhaan author
git config --global user.name "Dhamodharan"
git config --global user.email "dhamo@example.com"

# 2. Default branch name (GitHub default: main)
git config --global init.defaultBranch main

# 3. Line endings (Windows + Linux team)
git config --global core.autocrlf true

# 4. Default editor (VS Code)
git config --global core.editor "code --wait"

# 5. Verify everything
git config --list
```

Git Bash — verify config

```
$ git config --global user.name
Dhamodharan
$ git config --global user.email
dhamo@example.com
$ git config --list --show-origin
file:C:/Users/dhamo/.gitconfig
  user.name=Dhamodharan
  init.defaultbranch=main
```

.gitconfig — Visual Studio Code

```
[user]
    name = Dhamodharan
    email = dhamo@example.com
[init]
    defaultBranch = main
[core]
    autocrlf = true
```

Fig 1.3 ILLUSTRATIVE MOCKUP **Global config file.** `~/.gitconfig` VS Code-la open panni verify pannalaam.

Common mistake 1 · Wrong email

Commits private email-la pogum, GitHub green squares varadhu. GitHub **Settings** → **Emails**-la verified email use pannunga. Check: `git config user.email`.

Common mistake 2 · Global vs local

Office laptop-la personal email set aagirukkum. Repo-specific-a fix panna: `git config user.email "office@company.com"` (`--global` illaama).

IMPORTANT NOTES

`--global` = all repos on your laptop. Repo-specific-a change panna `--global` remove pannunga. Config store aagura file: `C:\Users\<name>\.gitconfig`. `git config --list --show-origin` panna endha file-la irundhu value vandhadhu nu theriyum — office-la proxy/SSL issues debug panna useful.

1.8.1 Config levels — which one wins?

Level	Flag	File	Scope
System	`--system`	`C:\Program Files\Git\etc\gitconfig`	All users on the machine
Global	`--global`	`C:\Users\<name>\.gitconfig`	Your user, all repos
Local	`--local`	`.git/config`	This repo only (highest priority)

1.9 Create Your First Repository

Repository (repo) na project folder + Git history. Two ways create panna: (a) new folder-la `git init`, (b) GitHub-la irundhu `git clone` (Chapter 4). Ippo first method.

1. **Create folder & initialize**
   `mkdir -p ~/projects/git-practice` → `cd ~/projects/git-practice` → `git init`. `.git` nu hidden folder create aagum — adhu dhaan Git database. Delete pannadheenga.
2. **Create first file**
   `echo "# Git Practice" > README.md` or VS Code-la file create pannunga.
3. **Check status**
   `git status` — README.md untracked-a irukkaradhu paapenga.
4. **Stage & commit**
   `git add README.md` then `git commit -m "docs: add project readme"`.

git init — full first-repo flow

```
mkdir -p ~/projects/git-practice && cd ~/projects/git-practice
git init
echo "# Git Practice" > README.md
git status
git add README.md
git commit -m "docs: add project readme"
git log --oneline
```

Git Bash — expected output

```
$ git init
Initialized empty Git repository in C:/Users/dhamo/projects/git-practice/.git/
$ git status
On branch main · No commits yet
Untracked files:
        README.md
$ git add README.md
$ git commit -m "docs: add project readme"
[main (root-commit) 7f3a91c] docs: add project readme
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
```

Success check: `git log --oneline` output `7f3a91c docs: add project readme` maadhiri irundha first commit success. Indha hash dhaan commit ID — future-la idhu use panni version-ku return poga mudiyum.

REAL-TIME IT COMPANY USAGE

Company-la developer-ah join aana first day, TL `git clone` link kuduppaanga. But internal tools, scripts, and POCs-ku `git init` still daily use. New microservice create pannumbodhu team lead repo structure, `.gitignore`, README template ellam ready panni dhaan `git init` pannuvaanga.

1.10 Chapter Recap & Practice

- Git = distributed VCS; GitHub = cloud platform.
- Working directory → staging → local repo → remote.
- One-time setup: name, email, default branch.

PRACTICE TASK

Create `git-practice` folder, init Git, add 3 files (`index.html`, `style.css`, `README.md`), stage only two files, commit, and run `git status` + `git log`. Third file endhukku commit aagala nu explain pannunga.

CHAPTER 02 · DAILY TOOLKIT

Basic Git Commands

Ithu dhaan ungal daily driver — 13 commands, correct syntax, expected output, and real usage.

13 commands | Level: **Beginner** | Time: **90 min** | Practice: **Required**

Chapter 1-la `git init` and first commit pannom. Ippo complete command set paakkom. Indha 13 commands 90% daily work-ku enough — same repo-la practice pannunga.

2.1 git init — Start Tracking

terminal

```
git init                  # current folder-la new repo
git init my-app           # new folder create panni repo init
git init --bare repo.git # server-side bare repo (no working files)
```

What it does: `.git` hidden folder create pannum — adhula objects, refs, config, hooks ellam irukkum. Init panna odane files commit aagala — `git add` + `git commit` venum.

2.2 git status — Your Best Friend

Indha command use pannina current repository-la enna files modified, staged, untracked-nu therinjukka mudiyum. Idhu safe command — edhuvum change pannadhu.

terminal

```
git status            # full status
git status -s         # short status (M, A, ??)
git status --branch   # branch + ahead/behind info
```

Full status — readable

```
$ git status
On branch main
Changes not staged for commit:
  modified:   index.html

Untracked files:
  style.css

no changes added to commit
```

STATUS CODES

`M` modified · `A` added · `D` deleted · `??` untracked · `UU` merge conflict. Short form: `git status -s`. Upstream info: `git status --branch`.

2.3 git add — Stage Your Changes

Commit panna munnaadi changes-ah staging area-ku add pandrom. Idhu "select" step. Indha command illaama new file commit aagadhu.

terminal

```
git add index.html          # one file
git add .                   # all changes in current folder
git add -A                  # all changes incl. deletes
git add -p                  # choose hunks interactively
```

COMMON MISTAKE

`git add .` panna `node_modules/`, `.env`, build output ellam stage aagum. First `.gitignore` create pannunga (Section 2.10), appuram add pannunga. Password/API key commit aana GitHub history-la irundhu remove panna romba kashtam.

2.3.1 VS Code Source Control Panel

Terminal type pannaama kooda staging and commit pannalaam. VS Code-la left side **Source Control** icon click pannunga (or `Ctrl + Shift + G`). Changes list, stage (+), discard, and commit message box ellam inga irukku. Beginners-ku idhu easiest.

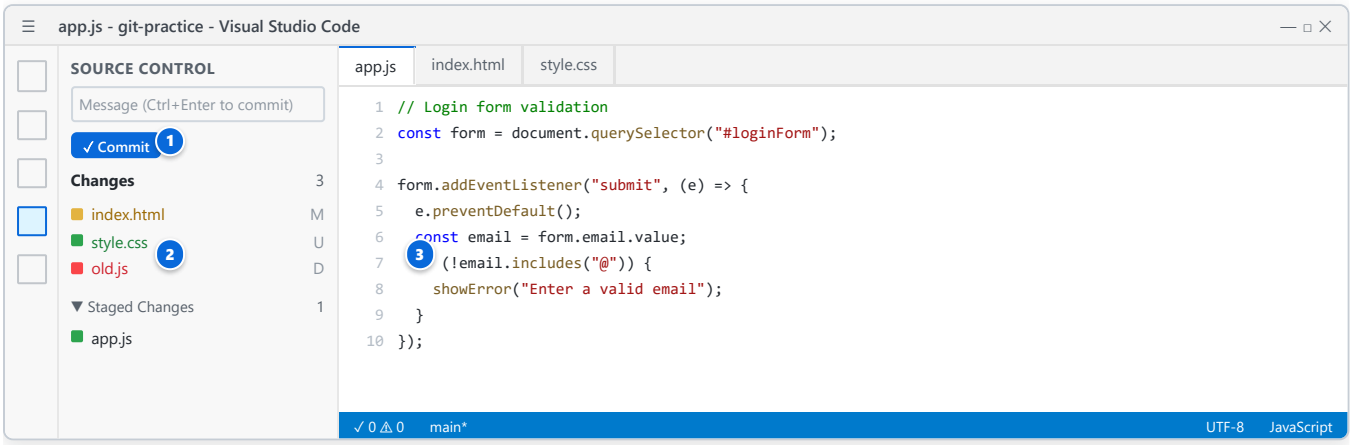


Fig 2.1 ILLUSTRATIVE MOCKUP **VS Code Source Control panel.** ① Activity bar Source Control icon. ② Commit message box + Commit button. ③ Changed files with status letters (M/U/D). + click pannina stage aagum.

2.4 git commit — Save a Snapshot

Staged changes-ah local repository-la permanent snapshot-ah save pannum. Every commit-ku unique ID (hash), author, date, message irukkum. Commit = project photo album-la oru photo madhiri.

terminal

```
git commit -m "feat: add login form validation"
git commit -am "fix: correct email regex"   # tracked files only
git commit --amend -m "better message"      # fix last commit
git commit --allow-empty -m "chore: trigger CI"
```

Git Bash — commit output

```
$ git commit -m "feat: add login form validation"
[main 4b8e2f1] feat: add login form validation
 2 files changed, 34 insertions(+), 2 deletions(-)
 create mode 100644 js/login.js
```

Good commit message

`feat: add login form validation`
`fix: handle null user in profile API`
`docs: update setup instructions`

Bad commit message

`update` `changes` `final code` `asdf`
Reviewer-ku enna change nu theriyadhu, future-la blame/bisect useless aagum.

Rule: one logical change = one commit. "Login page + API refactor + CSS fix" ellam ore commit-la panna review panna kashtam. Type prefix (`feat`, `fix`, `docs`, `test`, `chore`) team convention — Chapter 12-la detail.

Part	Example	Meaning
Type	`feat`, `fix`, `docs`	What kind of change
Scope (optional)	`feat(auth):`	Which module
Subject	`add JWT login`	Imperative, under 50 chars

2.5 git log — Read History

Commit history paakka. Default output full details; practical-a oneline + graph options dhaan daily use.

terminal

```
git log                                 # full history
git log --oneline                       # one line per commit
git log --oneline --graph --all         # branch graph
git log -5                              # last 5 commits
git log --author="Dhamodharan"
git log --since="2 weeks ago" --oneline
git log -p index.html                   # changes + history
```

Git Bash — git log --oneline --graph --all

```
* 9c1d4ab (HEAD -> main, origin/main) merge: feature/login into main
|\
| * 4b8e2f1 (feature/login) feat: add login form validation
| * 1a7c002 feat: create login page UI
|/
* 7f3a91c docs: add project readme
```

Options: `--oneline` short hash · `--stat` files changed · `-p` full diff · `--author` / `--since` filter.

2.6 git diff — Compare Changes

Commit panna munnaadi "enna maathina?" nu verify panna. Code review-ku munnaadi indha habit vachukonga.

terminal

```
git diff                # working dir vs staging
git diff --staged       # staging vs last commit
git diff HEAD           # working dir vs last commit
git diff main..feature  # between two branches
git diff --stat         # summary only
```

Git Bash — git diff

```
diff --git a/index.html b/index.html
@@ -12,6 +12,8 @@
   <h1>Welcome</h1>
-  <p>Old text</p>
+  <p>Updated text</p>
+  <button>Login</button>
```

HOW TO READ A DIFF

Red / minus = removed line. **Green / plus** = added line. `@@ -12,6 +12,8 @@` = old file line 12-la 6 lines, new file line 12-la 8 lines. `a/` = old version, `b/` = new version.

2.7 git show — Inspect One Commit

terminal

```
git show                    # latest commit details
git show 4b8e2f1            # specific commit
git show --stat 4b8e2f1     # files changed only
git show 4b8e2f1:index.html # file content at that commit
```

Use case: QA "yesterday login page break aachu" nu sonna, `git log -- js/login.js` panni last change commit ID edunga, `git show <id>` panni exactly enna maathina nu paarunga. `git blame file` panna line by line who changed last nu kooda theriyum.

2.8 git rm — Delete & Stage

terminal

```
git rm old.js              # file delete + stage
git rm -r assets/          # folder delete
git rm --cached .env       # stop tracking, keep local file
```

`git rm` = file system-la irundhu delete + staging area-la deletion record. `rm old.js` (normal delete) panna `git add -A` venum. **Most important:** `.env` accidentally commit aana `git rm --cached .env` use pannunga — local file safe-a irukkum, Git tracking mattum stop aagum.

2.9 git mv — Rename & Move

terminal

```
git mv old-name.js new-name.js
git mv src/utils.js src/helpers/utils.js
```

Idhu rename + stage rendum ore step-la pannum. Git internally rename-ah detect pannum (delete + add madhiri irundhalum history follow pannum). Normal rename pannina `git add -A` pannunga.

2.10 .gitignore — Keep Noise Out

Git ignore pannavendiya files-ah project root-la `.gitignore` file-la list pandrom. Idhu romba important — secrets, dependencies, build output repo-la poga koodadhu.

.gitignore

```
# dependencies
node_modules/
target/
vendor/

# secrets — NEVER commit
.env
.env.local
*.pem
application-prod.properties

# build output
dist/
build/
*.class
*.log

# IDE / OS
.vscode/
.idea/
.DS_Store
Thumbs.db

# keep folder but ignore contents
uploads/*
!uploads/.gitkeep
```

Pattern	Meaning
`*.log`	All .log files, any folder
`build/`	build directory
`**/temp`	temp in any subfolder
`!keep.txt`	Exception — don't ignore
`/config.js`	Root-la mattum

ALREADY COMMITTED?

`.gitignore` add pannina future changes mattum ignore aagum. Already tracked file-ku: `git rm --cached file` → commit. `node_modules` commit aana repo size explode aagum.

Pro tip: GitHub repo create pannumbodhu language-based .gitignore template select pannalaam (Node, Java, React). `git check-ignore -v .env` panna endha pattern ignore pandradhu nu kaattum; `git status --ignored` panna ignored files list kaattum. Company-la usually shared template irukkum — adhai copy pannunga.

2.11 git restore — Undo Local Changes

terminal

```
git restore index.html              # discard working changes (danger!)
git restore --staged index.html     # unstage, keep changes
git restore --source=HEAD~1 app.js  # old version content
```

DATA LOSS WARNING

`git restore index.html` panna ungal unsaved edits permanently poidum — Git-la irundhu recover panna mudiyaadhu. Commit pannaadha work-ah discard pannumbodhu double-check pannunga. Safe alternative: `git stash` (Chapter 6).

2.12 git clean — Remove Untracked Files

terminal

```
git clean -n          # dry run — enna delete aagum nu kaattum
git clean -f          # untracked files delete
git clean -fd         # untracked files + folders
git clean -fdx        # include ignored files (node_modules!)
```

Always `-n` first. Idhu untracked files-ah mattum remove pannum; Git restore panna mudiyaadhu. `-x` use pannina `.env`, `node_modules` kooda poidum — build errors varum.

2.13 git config Aliases — Work Faster

terminal

```
git config --global alias.st "status -s"
git config --global alias.co "checkout"
git config --global alias.br "branch"
git config --global alias.lg "log --oneline --graph --all"
# usage
git st        # same as: git status -s
git lg        # beautiful history graph
```

Situation	Correct command
Edited file, not staged yet	`git restore file` (discard) or `git stash` (keep)
Staged by mistake	`git restore --staged file`
Untracked junk files	`git clean -fd` (dry run `-n` first)
Bad commit, not pushed	`git reset --soft HEAD~1` or `--mixed`
Bad commit, already pushed	`git revert <hash>` — never reset shared branch

CHAPTER RECAP & PRACTICE

Daily flow: `git status` → edit → `git diff` → `git add -p` → `git commit -m "..."` → `git log --oneline`. **Practice:** repo-la 3 files create panni, oru file .gitignore-la add panni, staged/unstaged diff paathu, 2 separate commits pannunga. Appuram `git restore --staged` and `git clean -n` try pannunga.

CHAPTER 03 · YOUR CLOUD HOME

GitHub Repository Management

Account create pannuvom, repository-ah correct-a configure pannuvom — README, license, visibility, releases varaikkum.

11 topics | Level: **Beginner** | Time: **60 min** | Needs: **GitHub account**

Git lokal-a work aagum; GitHub-la ungal code-ku oru **professional home** create aagum. Recruiters ungal GitHub profile paathu dhaan shortlist pandranga. So repo name, README, license, and settings ellam serious-a take pannunga.

3.1 Create Your GitHub Account

1. **Sign up**
   github.com → Sign up → username, email, strong password. Username professional-a vachukonga (`dhamo-dev`), not `coolboy2005`.

2. **Verify email**
   Inbox-la verification link click pannunga. Idhu illaama Pull Requests, Actions ellam block aagum.

3. **Profile setup**
   Settings → Public profile: full name, bio, location, company, website, profile picture. Recruiter-ku idhu dhaan first impression.

4. **Enable 2FA**
   Settings → Password and authentication → Two-factor authentication. Company org-ku contribute panna mandatory (Chapter 9).

3.2 Create a Repository

Top-right **+** icon → **New repository**. Indha fields-ah carefully fill pannunga.

Field	What to choose
Repository name	lowercase + hyphens: `employee-management`. Spaces/underscores avoid pannunga.
Description	One line: "React + Spring Boot employee CRUD app". Repo card-la idhu dhaan kaattum.
Visibility	**Public** = portfolio/open source. **Private** = company/internal work.
Initialize	New project-ku README + .gitignore + license add pannunga. Existing local repo-ku **tick pannadheenga**.

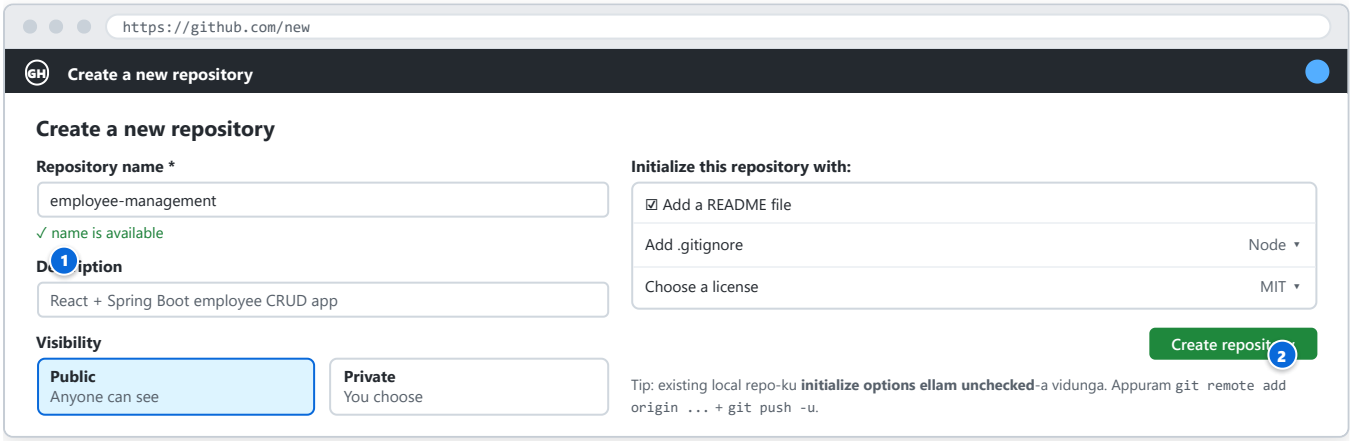


Fig 3.1 ILLUSTRATIVE MOCKUP **New repository page.** ① Name + visibility (public/private). ② Create button — initialization options select pannunga.

3.3 Public vs Private Repository

Interview-la common question. Free plan-la unlimited public and private repos irukku; difference access-la mattum.

Aspect	Public	Private
Who can view	World (anyone with link)	Only you + invited collaborators
Who can clone	Anyone	Members with access
Actions minutes	Free (standard runners)	Free quota, then billed
Pages hosting	Free	Paid plans (Pro/Team)
Best for	Portfolio, open source, learning	Company code, client work, secrets
Security	Everyone sees code + history	Controlled; still rotate secrets

GOLDEN RULE

Public repo-la **never** commit `.env`, API keys, database passwords, or client data. Private repo kooda safe illa — secrets-ah always GitHub Secrets-la vachukonga.

3.4 README.md — Repo Home Page

Repo open pannina first-a README.md content dhaan kaattum. Idhu ungal project-oda **front page + documentation + recruiter pitch**.

README.md

```
# Employee Management System

React + Spring Boot CRUD application with JWT login.

## Tech Stack
React 18 · Spring Boot 3 · MySQL 8 · JWT
## Features
Employee CRUD, JWT authentication, role-based access.

## Run Locally
```bash
git clone https://github.com/dhamo-dev/employee-management.git
cd backend && mvn spring-boot:run
cd ../frontend && npm install && npm run dev
```

Author
Dhamodharan – [LinkedIn](https://linkedin.com/in/...)
```

Must have: project name + one-line purpose, tech stack, setup commands, live demo, screenshots, folder structure, author contact. **Nice to have:** build/license badges, API table, architecture diagram, roadmap checklist.

3.5 LICENSE — Legal Permission

License illaama code "all rights reserved" — yaarum legally use/copy panna mudiyaadhu. Open source-ku license must. Company internal repos-ku license usually vendaam.

License	You allow	Common use
MIT	Use, modify, sell — keep copyright notice	Most beginner/portfolio projects
Apache 2.0	Same as MIT + patent protection	Enterprise-friendly open source

3.6 .gitignore Templates

`Repo create pannumbodhu` **Add .gitignore** dropdown-la language select pannina GitHub automatically template create pannum. Illa-na repo root-la manual-a file create pannunga.

Node
`node_modules/`
`dist/`
`.env`

Java
`target/`
`*.class`
`.idea/`

OS / IDE
`.DS_Store`
`Thumbs.db`
`.vscode/`

3.7 Repository Settings (Important Tabs)

Setting	What it controls
General	Repo name (rename), default branch, description, topics, archive/delete.
Collaborators	People + permission level (Read / Triage / Write / Maintain / Admin).
Branches	Branch protection rules: required reviews, status checks, no force-push.
Actions	Workflow permissions, secrets, runner access.
Pages	Static site hosting config (Chapter 11).
Danger Zone	Transfer ownership, archive (read-only), delete permanently.

DISCOVERY TIP · TOPICS
Settings → General → **Topics**-la `react`, `spring-boot`, `jwt` add pannunga — search-la project discover aagum.

3.8 Watchers, Stars & Forks

★ Star
Like/save. Bookmark maadhiri — ungal starred repos profile-la kaattum. Popularity signal.

◎ Watch
Repo notifications. All activity, issues only, or ignore. Company repos-ku issues-only set pannunga.

⩥ Fork
Someone else's repo-va ungal account-ku copy. Open source contribution-ku first step (Chapter 7).

Note: star/fork = independent copy; changes auto-sync aagadhu. Original repo-ku contribution panna Pull Request venum (Chapter 7).

3.9 Repository Description & Social Preview

Description (one line) + website URL + topics fill pannunga. Settings → General → **Social preview**-la 1280×640 image upload pannina LinkedIn/Twitter share professional card kaattum.

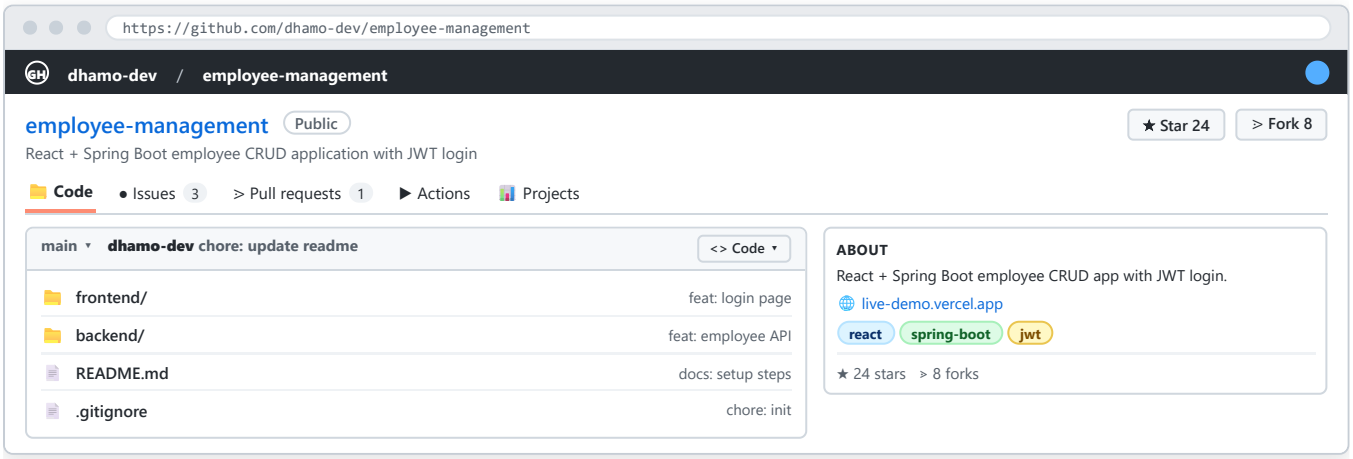


Fig 3.2 ILLUSTRATIVE MOCKUP **Repository home page.** README, topics, description, stars, and file list — recruiter first impression.

3.10 Releases & Tags

Deployment-ready version-ah **release** ah publish pandrom. Internally idhu Git **tag** (version label) + release notes. QA/Production-ku "v1.2.0" nu fixed version kuduppom.

terminal

```
git tag -a v1.0.0 -m "First production release"
git push origin v1.0.0

# later releases
git tag -a v1.1.0 -m "Add employee search"
git push origin --tags
```

GitHub UI: repo → **Releases** → **Draft a new release** → tag select → title + release notes → attach build files (.jar, .zip) → Publish. Release notes-la features, bug fixes, breaking changes list pannunga.

Tag
Git-level pointer to a commit. `v1.0.0`

Release
GitHub-level package: tag + notes + binaries.

SemVer
`MAJOR.MINOR.PATCH` — 1.2.3 (Chapter 12).

3.11 Transfer, Archive & Delete

Action	What happens	When to use
Transfer	Repo new owner-ku (user/org) pogum; URL change aagum; stars/forks/issues carry over.	Personal project org-ku move panna; team ownership change.
Archive	Read-only aagum. Issues/PRs/comments freeze; clone still possible.	Old project maintain pannaadha; reference-ku vachukka.
Delete	Repo + issues + PRs + releases permanently delete. 90 days recovery only via support (sometimes).	Duplicate/dummy repos clean panna — double confirm.

BEFORE DELETE
`Settings → General → Danger Zone`. Type repo name to confirm. Local clone irundha `.git` history kooda irukku, but GitHub issues/PRs/releases poidum. Team repo-ah delete pannaadheenga — archive pannunga.

REAL-TIME IT COMPANY USAGE
Company-la repo naming convention strict-a irukkum: `team-project-service` (ex: `payments-invoice-api`). README-la mandatory sections: owner team, on-call contact, setup, deploy steps, and architecture link. Branch protection + CODEOWNERS + release tags ellam repo create pannumbodhe configure pannuvaanga. Recruiters/interviewers ungal personal repo-la README, commit frequency, and project structure paathu evaluate pandranga.

3.12 Chapter Recap & Practice

- Repo name lowercase-hyphen; description + topics fill pannunga.
- Public = portfolio, Private = company code.
- README = setup steps + screenshots + tech stack.
- License (MIT) open source-ku; releases = tag + notes.

PRACTICE TASK
Ungal first portfolio repo create pannunga: README with 3 sections, MIT license, Node .gitignore, 5 topics, and one release `v0.1.0`. Appuram repo Settings ellam open panni endha option enna pannudhu nu note pannunga.

CHAPTER 04 · CONNECT TO GITHUB

Remote Repository

Local code-ah GitHub-ku connect pannuvom — push, pull, fetch, clone, and SSH setup.

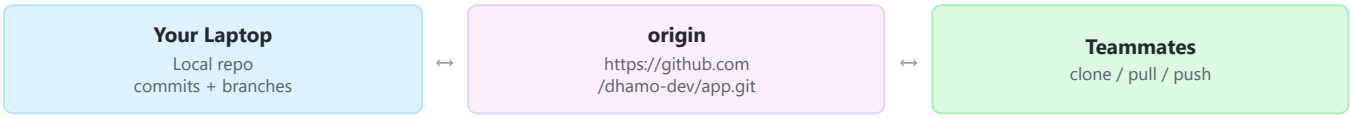
11 topics | Level: **Beginner** | Time: **75 min** | Needs: **GitHub repo**

Chapter 3-la GitHub-la empty repo create pannom. Ippo ungal laptop-la irukkura code-ah adhoda connect pannanum. Indha bridge dhaan **remote**. Remote illaama team collaboration illa, backup illa, CI/CD illa.

4.1 What is a Remote Repository?

DEFINITION

Remote repository na ungal local repo-oda copy, oru server-la (GitHub) irukku. Git-la oru repo-ku multiple remotes irukkalaam; default name `origin`. Remote = bookmarked URL + tracking branches.



Key idea: commit pannina changes local-la mattum irukkum. `git push` panna dhaan GitHub-ku pogum. Teammates `git pull` panna dhaan avangalukku varum. Remote automatic-a sync aagadhu.

4.2 git remote add — Connect Local to GitHub

terminal

```
# GitHub repo page-la irundhu URL copy pannunga
git remote add origin https://github.com/dhamo-dev/git-practice.git
git push -u origin main
```

`origin` = remote-ku namma kudukkura nickname (any name allowed, but origin convention). `-u` = upstream set pannum; next time `git push` mattum podhum. **Existing repo with commits** irundha GitHub-la repo create pannumbodhu README/gitignore initialize pannaadheenga — conflict varum.

4.3 git remote -v / rename / remove

terminal

```
git remote -v                        # list remotes with URLs
git remote show origin               # branches + tracking details
git remote rename origin upstream    # rename
git remote set-url origin git@github.com:dhamo-dev/app.git
git remote remove origin             # disconnect (local repo stays)
```

Git Bash — git remote -v

```
$ git remote -v
origin  https://github.com/dhamo-dev/git-practice.git (fetch)
origin  https://github.com/dhamo-dev/git-practice.git (push)
$ git remote show origin
* remote origin
  Fetch URL: https://github.com/dhamo-dev/git-practice.git
  HEAD branch: main
```

Common use: repo transfer aana apparam `set-url`; fork workflow-la `origin` = your fork, `upstream` = original repo (Chapter 7).

4.4 git push — Upload Commits

Local commits-ah remote-ku anuppum. Push panna munnaadi `git pull` pannina conflict avoid aagum (team repo-la must).

terminal

```
git push                            # upstream set aana apparam
git push -u origin main             # first push
git push origin feature/login       # specific branch
git push --force-with-lease         # safe force (never on main)
git push origin --delete old-branch # delete remote branch
```

Git Bash — first push output

```
$ git push -u origin main
Enumerating objects: 24, done.
Writing objects: 100% (24/24), 6.42 KiB | 1.20 MiB/s, done.
To https://github.com/dhamo-dev/git-practice.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

NEVER FORCE-PUSH SHARED BRANCHES

`git push --force` panna remote history overwrite aagum; teammates commits lose aagalaam. Main/develop-la force push totally banned. Unavoidable-a irundha `--force-with-lease` mattum use pannunga.

4.5 git pull — Download + Merge

`git pull` = `git fetch` + `git merge`. Remote changes-ah download panni current branch-la merge pannum. Daily standup-ku munnaadi idhu pannunga.

terminal

```
git pull                    # fetch + merge (default)
git pull --rebase           # fetch + rebase (clean linear history)
git pull origin develop     # specific branch
```

Git Bash — git pull

```
$ git pull
remote: Enumerating objects: 7, done.
From https://github.com/dhamo-dev/git-practice
   4b8e2f1..9c1d4ab  main       -> origin/main
Updating 4b8e2f1..9c1d4ab
Fast-forward
 src/App.jsx | 12 ++++++------
 1 file changed, 6 insertions(+), 6 deletions(-)
```

PULL REJECTED?

`error: Your local changes would be overwritten by merge` vandha, local changes commit pannunga or `git stash` pannunga. Appuram `git pull`. Conflict vandha `git status` panni files fix pannunga (Chapter 5).

4.6 git fetch — Download Only

`fetch` panna ungal working files touch aagadhu. `origin/main` pointer mattum update aagum. Enna varudhu nu safe-a paakka, appuram decide panna idhu best.

terminal

```
git fetch                # origin updates download, no merge
git fetch origin main    # one branch
git fetch --all --prune  # all remotes, delete stale refs
git log HEAD..origin/main --oneline   # what's new?
```

Git Bash — review before merge

```
$ git fetch origin
From https://github.com/dhamo-dev/employee-management
   7f3a91c..4b8e2f1  main -> origin/main
$ git log HEAD..origin/main --oneline
4b8e2f1 feat: add login form validation
1a7c002 feat: create login page UI
$ git merge origin/main
Updating 7f3a91c..4b8e2f1
Fast-forward
```

4.7 git fetch vs git pull

Point	git fetch	git pull
What it does	Downloads remote commits only	Downloads + merges into current branch
Working files	Not touched	Changed immediately
Merge conflicts	No (nothing merged)	Possible
Safe?	Always safe	Depends on local changes
Use when	Review first, then merge manually	You want latest code now

4.7.1 The safe sync workflow (recommended)

1. **Fetch first**
   `git fetch origin` — enna varudhu nu paakka, files safe.
2. **Review incoming changes**
   `git log HEAD..origin/main --oneline` and `git diff HEAD origin/main`.
3. **Decide**
   Problem illa-na `git merge origin/main` (or `git pull`). Local changes irundha commit/stash pannunga.

Thing	Syncs automatically?	How
Commits	No	`git push` / `git pull`
Branches	No	`git push -u origin branch`
Tags	No	`git push --tags` / `git fetch --tags`
Stashes	Never	Local only — remote-ku pogaadhu
Untracked files	Never	Commit panni dhaan share panna mudiyum
Remote URL config	No	`git remote set-url`

4.8 git clone — Copy a Repository

Remote repo-va full-a (history, branches, tags ellam) ungal laptop-ku copy pannum. Company-la first day idhu dhaan first command.

terminal

```
git clone https://github.com/dhamo-dev/employee-management.git
git clone git@github.com:dhamo-dev/employee-management.git   # SSH
```

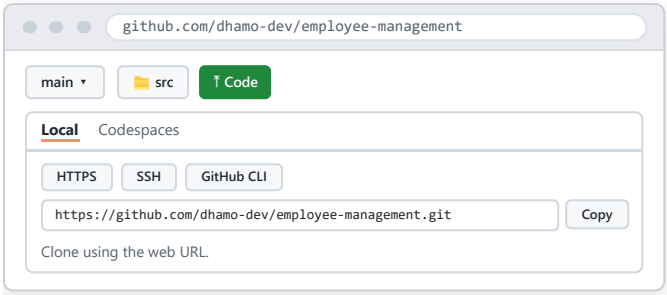


Fig 4.1 ILLUSTRATIVE MOCKUP **Code button → clone URL.** HTTPS or SSH tab select panni URL copy pannunga.

Git Bash — clone output

```
$ git clone https://github.com/dhamo-dev/employee-management.git
Cloning into 'employee-management'...
Receiving objects: 100% (186/186), 42.10 KiB
Resolving deltas: 100% (98/98), done.
$ cd employee-management
$ git remote -v
origin  https://github.com/dhamo-dev/employee-management.git
```

Clone vs ZIP: ZIP-la history illa, branch illa, push panna mudiyaadhu. Always clone pannunga.

4.9 Upstream Tracking Branches

`-u` (or `--set-upstream`) branch-ku remote partner-ah set pannum. Adhukku apparam `git push` / `git pull` commands-la remote/branch name type panna vendaam.

terminal

```
git push -u origin feature/login
git branch -vv                # tracking info (ahead/behind)
git branch --set-upstream-to=origin/main main
```

Git Bash — git branch -vv

```
$ git branch -vv
* main          9c1d4ab [origin/main] merge: feature/login
  feature/login 4b8e2f1 [origin/feature/login: ahead 2] login form
```

Read it: `ahead 2` = 2 local commits push panna pending. `behind 3` = 3 remote commits pull panna pending. `ahead 1, behind 2` = diverged — pull/rebase panni resolve pannunga.

AFTER CLONE — FIRST THREE COMMANDS

`git log --oneline -5` → recent history · `git branch -a` → local + remote branches · `git remote -v` → origin URL verify. Appuram `git switch -c feature/my-task` panni work start pannunga.

4.10 SSH vs HTTPS Authentication

GitHub-ku push/pull pannumbodhu identity prove pannanum. Rendu option irukku — beginners-ku HTTPS easy, daily developers-ku SSH better.

Point	HTTPS	SSH
URL format	`https://github.com/user/repo.git`	`git@github.com:user/repo.git`
Auth method	Personal Access Token (PAT)	SSH key pair
Setup	Easy — token generate pannunga	Moderate — keygen + add to GitHub
Password prompt	Every push (helper illaama)	No prompt after setup
Firewall/proxy	Works everywhere (port 443)	Port 22 block aagalaam
Best for	Beginners, CI, demo laptops	Daily development, multiple repos

SSH setup (one time)

```
ssh-keygen -t ed25519 -C "dhamo@example.com"
# Enter x3 (default path, empty passphrase)
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
cat ~/.ssh/id_ed25519.pub
# GitHub → Settings → SSH keys → New
ssh -T git@github.com
```

HTTPS TOKEN NOTE

GitHub password-ah push password-ah use panna mudiyaadhu (2021-la remove aachu). PAT create: Settings → Developer settings → Personal access tokens → Fine-grained token. Token-ah password-ah paste pannunga. **Token = password** — share pannaadheenga.

Git Bash — SSH test

```
$ ssh -T git@github.com
Hi dhamo-dev! You've successfully authenticated...
```

4.11 Credential Cache & Troubleshooting

terminal

```
# HTTPS credentials remember panna
git config --global credential.helper manager   # Windows
git config --global credential.helper cache     # 15 min memory
```

Error	Fix
`Password authentication was removed`	PAT use pannunga, password illa.
`Permission denied (publickey)`	SSH key GitHub-la add aagala; `ssh -T` test.
`remote origin already exists`	`git remote set-url origin <url>`.
`failed to push some refs`	Remote-la new commits; `git pull --rebase` then push.

REAL-TIME IT COMPANY USAGE

Company-la SSH keys mandatory; oru key pair, public key add pannuvanga. Offboard aana udane revoke. CI/CD-la OIDC use pandranga — long-lived PATs avoid.

4.12 Chapter Recap & Practice

Practice: empty repo create → local connect → `git push -u origin main` → GitHub-la edit & commit → `git log HEAD..origin/main` paarunga.

CHAPTER 05 · PARALLEL DEVELOPMENT

Branching & Merging

Branch create panni parallel-a work pannuvom, merge pannuvom, conflict-ah confident-a resolve pannuvom.

12 topics | Level: **Core skill** | Time: **90 min** | Most asked: **Interviews**

Idhu dhaan Git-oda real power. Branch = independent timeline; main-ah touch pannaama feature develop pannalaam. Company-la **main-la direct commit panna koodadhu** — ellaa work-um feature branch + PR vazhiya dhaan.

5.1 What is a Branch?

Branch na commit-ku oru movable pointer. `main` branch latest commit-ah point pannum. New branch create pannina Git files copy pannadhu — just oru 41-byte pointer file create pannum. Adhaan branching cheap and instant.

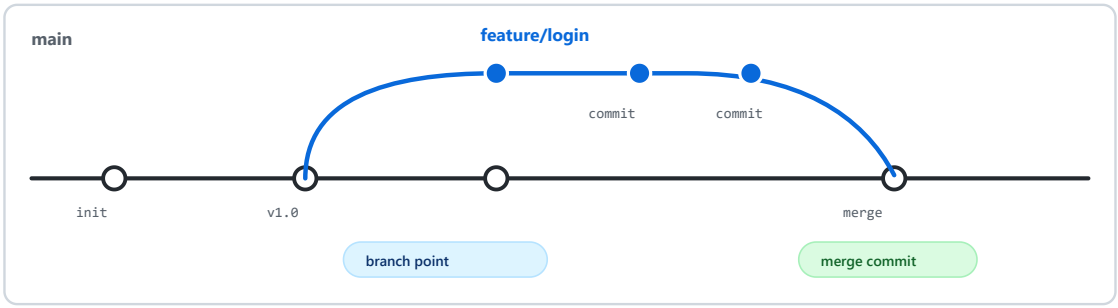


Fig 5.1 Branch = parallel timeline. Branch point-la irundhu feature commits separate-a pogum; merge-la thirumba main-ku join aagum.

5.2 git branch — Create & List

terminal

```
git branch                    # list local branches
git branch -a                 # + remote branches
git branch feature/login      # create (stay on current)
git branch -d feature/login   # delete (merged only)
git branch -D feature/login   # force delete (unmerged)
```

Git Bash — git branch

```
$ git branch
* main
  feature/login
  fix/navbar
```

5.3 git switch / git checkout

Branch create panni adhukku move aaga `git switch -c` (modern, recommended) or `git checkout -b` (old but everywhere).

terminal

```
git switch main                 # move to main
git switch -c feature/login     # create + move
git switch -                    # previous branch
git checkout -b hotfix/payment  # old style, same result
git checkout 4b8e2f1            # detached HEAD (view old commit)
```

UNCOMMITTED CHANGES IRUNDHA?

Switch panna mudiyadhu: `error: Your local changes would be overwritten`. Commit pannunga or `git stash`. Branch switch pannumbodhu uncommitted changes follow aagum — accidental-a wrong branch-la commit panna chance irukku.

5.4 Delete & Rename Branches

local branches

```
git branch -d feature/login     # safe delete
git branch -D feature/old       # force
git branch -m feature/auth      # rename current
git push origin --delete feature/login
```

RULE

Merge aana branch-ah mattum delete pannunga. `-d` refuse pannina, unmerged commits irukku — `git log main..feature/login` panni verify pannunga. Remote delete panna GitHub-la branch poidum, but PR history irukkum.

5.5 Merge — Fast-Forward

Feature branch create pannadhukku apparam main-la new commits illa-na, merge pannumbodhu Git pointer-ah just forward pannum. New merge commit create aagadhu. History straight line.

terminal

```
git switch main
git merge feature/login          # fast-forward if possible
git merge --no-ff feature/login  # always create merge commit
git merge --abort                # cancel a conflicted merge
```

Git Bash — fast-forward merge

```
$ git switch main
Switched to branch 'main'
$ git merge feature/login
Updating 7f3a91c..4b8e2f1
Fast-forward
 src/Login.jsx | 48 ++++++++++++++++++++
 1 file changed, 48 insertions(+)
```

5.6 Three-Way Merge

Main and feature rendum different commits advance aana, Git common ancestor + two branch tips (three points) use panni merge commit create pannum. Idhu dhaan **recursive / ort merge**.

Common ancestor 7f3a91c → **main tip** 9c1d4ab + **feature tip** 4b8e2f1 → **Merge commit** new commit, 2 parents

terminal

```
git switch main
git pull                         # latest main
git merge feature/login
# output: Merge made by the 'ort' strategy.
```

Merge type	When	History
Fast-forward	Main-la new commits illa	Linear, no merge commit
Three-way	Both branches advanced	Merge commit with 2 parents
Squash	Many small commits → one	Linear, detailed history lost
Rebase	Replay commits on new base	Linear, no merge commit

5.7 What is a Merge Conflict?

Rendu branches **same file-la same lines** different-a change pannina, Git decide panna mudiyaadhu; appo conflict mark pannum. Idhu error illa — normal team workflow-la daily nadakkum, neenga dhaan final code decide pannanum.

Git Bash — conflict output

```
$ git merge feature/login
Auto-merging src/App.jsx
CONFLICT (content): Merge conflict in src/App.jsx
Automatic merge failed; fix conflicts and then commit the result.
```

When conflicts happen
Same line edited in both branches · file deleted in one, edited in other · rename + modify · binary files changed.

When they don't
Different files · different lines in same file · one branch only added new content below.

5.8 Resolve a Conflict — Step by Step

1. **Check status**
   `git status` — `both modified: src/App.jsx` nu kaattum. `Unmerged paths` list paarunga.

2. **Open the file**
   VS Code-la markers visible aagum: `<<<<<<< HEAD`, `=======`, `>>>>>>> feature/login`. Inline buttons: **Accept Current**, **Incoming**, **Both**.

3. **Choose the final code**
   Rendum venum-na manually combine pannunga. Markers-ah **fully remove** panni test pannunga.

4. **Stage & commit**
   `git add src/App.jsx` → `git commit`. Git default merge message suggest pannum; meaningful message add pannalam.

5. **Verify**
   `git log --oneline --graph -5` and app-ah run panni functionality check pannunga. Appuram branch delete pannunga.

src/App.jsx — conflict markers

```
<<<<<<< HEAD
  <h1>Welcome back</h1>
=======
  <h1>Welcome, {user.name}</h1>
>>>>>>> feature/login
  <LoginForm />   // code continues
```

GOOD RESOLUTION
Rendu intention-um merge pannunga: `<h1>Welcome back, {user.name}</h1>`. Markers illaama, app run aagi, tests pass aana dhaan commit.

NEVER DO THIS
Markers-ah code-la vidama commit pannaadheenga. `--ours` / `--theirs` blindly use pannaadheenga — other side work lose aagum.

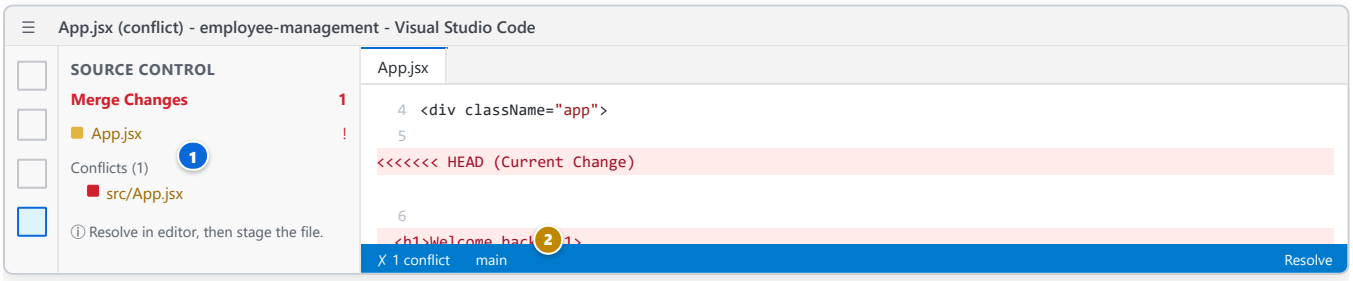


Fig 5.2 ILLUSTRATIVE MOCKUP **VS Code merge conflict view.** ① Conflicted file in Source Control. ② Current vs Incoming changes — Accept buttons use panni final code select pannunga.

5.9 git rebase — Move Commits to a New Base

Feature branch commits-ah main-oda latest commit mela replay pannum. Result: straight-line history, no merge commit. PR review-ku clean-a irukkum.

terminal

```
git switch feature/login
git fetch origin
git rebase origin/main
# conflict vandha
git add <file>
git rebase --continue
git rebase --abort          # cancel and go back
```

GOLDEN RULE OF REBASE

Never rebase shared/public branches. Rebase commit hashes-ah rewrite pannum; teammates history mismatch aagum. Rebase only **your own feature branch**. Push pannumbodhu `--force-with-lease` venum (normal push reject aagum).

5.10 git merge vs git rebase

Point	Merge	Rebase
History	Preserves real branching + merge commits	Rewrites into a linear history
Commit hashes	Unchanged	New hashes (rewritten)
Conflicts	Resolve once	Resolve per replayed commit
Safe on shared branch?	Yes	No — never
Best for	Merging PRs into main	Updating your feature branch

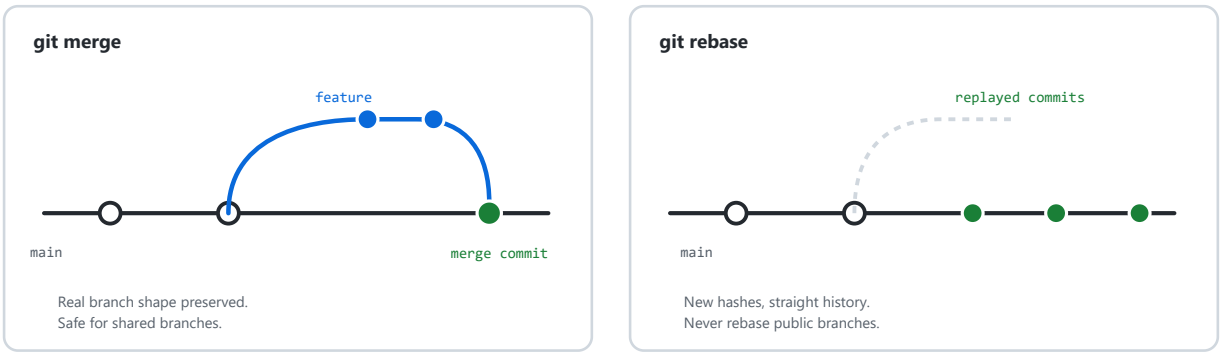


Fig 5.3 **Same work, two histories.** Merge keeps the branch shape and adds a merge commit; rebase replays your commits on top of main for a clean, linear log.

5.11 git cherry-pick — Copy One Commit

Oru branch-la irukkura specific commit-ah current branch-ku copy pannum. Hotfix main-ku mattum venum, feature branch-ku illa — appo idhu useful.

terminal

```
git switch main
git log --oneline feature/payment | head -5
git cherry-pick 8f2c1ab
# conflict vandha: fix → git add → git cherry-pick --continue
```

Git Bash cherry-pick

```
$ git cherry-pick 8f2c1ab
[main 3c9e7d2] fix: validate payment amount
 Date: Mon Nov 9 10:42:11 2026 +0530
 1 file changed, 4 insertions(+), 1 deletion(-)
```

Note: cherry-pick duplicate commit create pannum (same change, new hash). Long-term-la branches diverge aagum; emergency fix-ku mattum use pannunga.

5.12 Branching Strategies

Strategy	How it works	Best for
GitHub Flow	main + short-lived feature branches + PR + deploy	Web apps, startups, CI/CD teams
Git Flow	main, develop, feature/*, release/*, hotfix/*	Versioned releases, enterprise
Trunk-based	Everyone commits to main frequently, tiny branches	High-performing DevOps teams
Release branch	release/1.2 cut, only bug fixes, then merge to main	Mobile apps, quarterly releases

Team **Branch naming and lifecycle rules**

Naming convention

`feature/login-page` · `fix/navbar-overlap` · `hotfix/payment-500` · `chore/update-deps`. Lowercase, hyphen-separated, ticket ID optional: `feature/EMS-142-login`.

Lifecycle

Create from updated `main` → commit small → push daily → open draft PR early → review → merge → delete branch. Keep a branch alive under 1–3 days.

COMMON MISTAKES

Long-lived branches (merge pain grows daily) · branch from stale main · direct commits to main · deleting remote branches before PR merge · rebasing a branch that teammates already pulled.

REAL-TIME IT COMPANY USAGE

Most product teams GitHub Flow follow pandranga: `main` always deployable, feature branch 1–3 days max, PR review 1 approval minimum, merge pannina CI automatic-a deploy. Git Flow still used in banking/enterprise where monthly release irukku.

Practice: main-la 2 commits, feature branch-la 2 commits create pannunga. Fast-forward merge, then 3-way merge (main-la parallel commit) try pannunga. Same file same line edit panni conflict create panni resolve pannunga.

CHAPTER 06 · SAFE RECOVERY

Advanced Git

Mistake-ah undo panna, work-ah park panna, release-ku tag poda — professional developer tools.

16 topics | Level: **Intermediate** | Time: **120 min** | Most asked: **Scenarios**

Idhu dhaan beginner-ku senior difference. Code wrong-a commit aachu, branch clean panna venum, production-ku version tag poda venum — indha chapter-la ellam step-by-step paakkom. Ellam **local repo-la safe-a practice** pannalaam.

6.1 git stash — Park Unfinished Work

Half-done code-ah commit pannaamal temporarily side-la vekka. Urgent hotfix vandha, current feature-ah stash panni, main-ku switch panni fix pannalaam, appuram thirumba apply pannalaam. **Stash local-only** — push aagadhu, teammates-ku theriyaadhu.

terminal

```
git stash                       # save tracked changes
git stash push -m "login wip"   # with a message
git stash -u                    # include untracked files
git stash list                  # see all stashes
git stash show -p stash@{0}     # inspect diff
```

Git Bash — stash

```
$ git stash push -m "login wip"
Saved working directory and index state On main: login wip
$ git stash list
stash@{0}: On main: login wip
```

6.2 stash apply / pop / drop

terminal

```
git stash apply             # restore, keep in stash list
git stash pop               # restore and remove from list
git stash apply stash@{2}   # pick a specific stash
git stash drop stash@{0}    # delete one
git stash clear             # delete all (careful)
```

COMMON MISTAKE

`stash pop` conflict aana stash automatically delete aagadhu — resolve panni `git stash drop` pannunga. Stash-ah backup-nu nenachaadheenga: branch delete/rebase pannumbodhu confusion varum. Long-term work-ku WIP commit or draft PR better.

STASH VS WIP COMMIT

Stash = private, temporary, no history. WIP commit = branch-la visible, push pannalaam, teammates review pannalaam. One hour work-ku stash; multi-day work-ku WIP commit + draft PR better.

6.3 git reset — Move HEAD Back

`reset` branch pointer-ah old commit-ku move pannum. Evalavu changes keep pannanum-nu **mode** decide pannum. Shared branch-la history rewrite aagum — already pushed commits-ku `reset` use pannaadheenga; `revert` use pannunga.

Mode	HEAD moves	What happens to changes
`--soft`	Yes	Keep staged (ready to commit again)
`--mixed` (default)	Yes	Keep in working dir, unstaged
`--hard`	Yes	**Discard everything** — dangerous

terminal

```
git reset --soft HEAD~1      # undo commit, keep files staged
git reset HEAD~1             # undo commit + unstage
git reset --hard HEAD~1      # throw away last commit + changes
git reset --hard origin/main # match remote exactly
```

DANGER ZONE

`--hard` panna uncommitted work permanently poidum. First `git stash` pannunga or `git reflog` note pannunga. Unpushed commit-ah recover panna reflog help pannum; untracked files-ku reflog help pannadhu.

6.4 git revert — Safe Undo on Shared Branch

Old commit-ah delete pannaama, adhukku **opposite new commit** create pannum. History preserve aagum, adhaala main branch-la safe. Production bug commit-ah undo panna idhu dhaan correct method.

terminal

```
git revert 8f2c1ab             # revert one commit
git revert HEAD                # revert latest
git revert 8f2c1ab..3c9e7d2    # range (careful with order)
git revert --no-edit 8f2c1ab   # accept default message
```

Git Bash revert

```
$ git revert --no-edit 8f2c1ab
[main 7d1a0c4] Revert "fix: validate payment amount"
 1 file changed, 1 insertion(+), 4 deletions(-)
```

Map Which undo command should I use?

Situation	Correct command
Uncommitted file change	`git restore file.js`
Staged but not committed	`git restore --staged file.js`
Last commit, not pushed	`git reset --soft HEAD~1`
Commit already pushed	`git revert <hash>`

6.5 git reflog — Your Safety Net

Reflog HEAD move aana ellaa action-um record pannum: commit, reset, rebase, merge, checkout. Mistake panna 30 days varaikkum (default) recover pannalaam. **Git-la edhuvum truly lost illa** — reflog theriyadhu dhaan problem.

terminal

```
git reflog                    # every HEAD movement
git reflog show feature/login  # branch-specific
git reset --hard HEAD@{3}      # go back to entry 3
git branch rescue HEAD@{5}     # create branch at old state
```

Git Bash — git reflog

```
$ git reflog
7d1a0c4 HEAD@{0}: reset: moving to HEAD~1
8f2c1ab HEAD@{1}: commit: fix: validate payment
4b8e2f1 HEAD@{2}: checkout: moving from main to feature/login
```

RECOVERY RECIPE

Wrong `reset --hard` pannitinga? `git reflog` open panni lost commit hash-ah kandupidinga, appuram `git reset --hard <hash>` or `git branch rescue <hash>`. Reflog-ah first check pannunga — panic aaga vendaam.

6.6 git tag — Mark a Release

Commit-ku permanent name kuduthu release mark pannalaam: `v1.0.0`, `v1.2.3`. Branch move aagum, tag move aagadhu. CI/CD-la tag push aana automatic production deploy trigger aagum.

terminal

```
git tag v1.0.0                          # lightweight
git tag -a v1.0.0 -m "First stable release" # annotated
git tag                                 # list local tags
git show v1.0.0                         # tag details
git push origin v1.0.0                  # push one tag
git push origin --tags                  # push all tags
git tag -d v1.0.0                       # delete local tag
```

SEMVER REMINDER

`MAJOR.MINOR.PATCH` → breaking change / new feature / bug fix. Annotated tag (`-a`) always prefer pannunga — tagger name, date, message store aagum. Tag delete panna remote-la separate-a `git push origin --delete v1.0.0` venum.

1. **Tag the release commit**
   `git tag -a v1.4.0 -m "Release 1.4.0"` — main-la merge aana exact commit-la tag podunga.
2. **Push the tag**
   `git push origin v1.4.0`. Tag push panna CI/CD production deploy trigger aagum.
3. **Create GitHub Release**
   Repo → Releases → Draft a new release → tag select → auto-generated notes → Publish.
4. **Attach artifacts**
   Build zip, checksum, migration notes attach pannunga. Release page = customers-ku official changelog.

6.7 git commit --amend — Fix the Last Commit

Last commit message typo, forgotten file, or small fix-ah same commit-la merge panna. **New hash** create aagum — already pushed commit-ah amend panna force push venum, so shared branch-la avoid.

terminal

```
git add forgotten-file.js
git commit --amend --no-edit      # keep same message
git commit --amend -m "feat: add login validation"
git commit --amend --author="Dhamo <dhamo@example.com>"
```

ONLY BEFORE PUSH

PR review-la "small fix" panna new commit add pannunga — amend + force push reviewer-ku confusion create pannum. Your own feature branch-la mattum amend use pannunga. After amend: `git push --force-with-lease`.

6.8 Interactive Rebase — Clean Your History

Push panna munnaadi commits-ah reorder, reword, squash, drop pannalaam. PR-la clean history reviewer-ku easy. `git rebase -i HEAD~3` last 3 commits-ah open pannum.

terminal

```
git rebase -i HEAD~3
# editor opens:
# pick   8f2c1ab feat: login form
# squash 3c9e7d2 fix typo
# reword 7d1a0c4 add validation
git rebase --continue
git rebase --abort
```

Command	Meaning
`pick`	Keep commit as-is
`reword`	Change commit message
`squash`	Merge into previous commit, edit message
`fixup`	Merge into previous, discard message
`drop`	Delete commit

6.9 Squash Merge

Feature branch-la 12 small commits irundhaalum, main-la **one clean commit** create pannalaam. PR merge button-la "Squash and merge" select pannunga, or terminal-la:

terminal

```
git switch main
git merge --squash feature/login
git commit -m "feat: add login page (#42)"
```

6.10 git worktree — Two Branches at Once

Same repo-la multiple folders, each different branch checkout. Urgent hotfix paakka current work-ah stash panna vendaam — separate worktree create pannunga. Build files, node_modules separate-a irukkum.

terminal

```
git worktree add ../hotfix-folder hotfix/payment
git worktree list
cd ../hotfix-folder
# fix, commit, push normally
git worktree remove ../hotfix-folder
git worktree prune
```

6.11 Git Hooks — Automate Checks

Commit/push nadakkumbodhu automatic script run aagum. `.git/hooks/` folder-la samples irukkum. Team-la share panna **Husky** (Node) or pre-commit framework use pandranga, because `.git/hooks` push aagadhu.

Hook	When it runs
pre-commit	Before commit created — lint, format, tests
commit-msg	Validate message format (Conventional Commits)
pre-push	Before push — unit tests, secret scan
post-merge	After merge — npm install, DB migrate

.husky/pre-commit

```
#!/bin/sh
npx lint-staged
npm test -- --watchAll=false
```

terminal

```
npm install --save-dev husky lint-staged
npx husky init
echo "npx lint-staged" > .husky/pre-commit
```

COMMON MISTAKE

Hooks-ah bypass panna `git commit --no-verify` use panna mudiyum — but CI-la same checks mandatory-a vekkanum. Hook checks-ah CI replace panna mudiyaadhu; rendum venum.

WORKTREE IN REAL LIFE

Production hotfix vandha: main worktree-la feature code irukkum, `../hotfix` worktree-la clean main checkout panni fix pannunga. IDE rendu window-la open pannalaam — same repo, different branches.

6.12 Git Submodules

Oru repo-va innoru repo-oda dependency-a include panna. Submodule = separate repo-oda specific commit-ku pointer. Monorepo illaama shared library use panna useful, but update/versioning complex.

terminal

```
git submodule add https://github.com/org/ui-kit.git libs/ui-kit
git submodule update --init --recursive   # after clone
git submodule update --remote             # latest remote
git submodule status
```

ALTERNATIVE

Modern teams package manager prefer pandranga: npm private package, Maven artifact, or monorepo. Submodule use pannina `.gitmodules` commit pannunga, and CI-la `--recursive` clone pannunga.

Option	Best for
Submodule	Exact commit pinning, separate release cycle
Subtree	Code copied into repo, simpler clone for juniors
Package	Versioned library via npm/Maven — preferred
Monorepo	Many packages, one CI, shared tooling

6.13 git bisect — Find the Bad Commit

Bug eppo vandhadhu theriyala? Binary search use panni `log2(n)` steps-la culprit commit kandupidikkalaam. 1000 commits = ~10 checks. Manual or automated.

terminal

```
git bisect start
git bisect bad                 # current version broken
git bisect good v1.2.0         # last known working tag
# test each checkout, then:
git bisect good   # or git bisect bad
git bisect reset               # back to original branch
```

Git Bash bisect

```
$ git bisect good
Bisecting: 6 revisions left to test after this (roughly 3 steps)
[a1b2c3d] feat: update checkout API
```

terminal

```
git bisect start
git bisect bad HEAD
git bisect good v1.2.0
git bisect run npm test -- --watchAll=false
git bisect reset
```

Automate: `git bisect run` — test pass aana good, fail aana bad automatic-a mark aagum. Exit code 125 = skip (flaky test).

6.14 git gc & prune — Clean the Repo

Git automatic-a garbage collection pannum, but manual-a force pannalaam. Unreachable objects compress/delete aagum. Disk space romba edutha repo-ku useful.

terminal

```
git gc                  # pack objects, clean garbage
git gc --aggressive     # slower, deeper cleanup
git prune               # delete unreachable objects
git count-objects -vH   # repo size summary
```

CAREFUL

`prune` panna unreachable commits permanently poidum — reflog recovery chance kammi. `gc` safe; `prune --expire now` dangerously aggressive. Large binary history unglikely repo-ku Git LFS migrate better.

6.15 Git LFS — Large Files

Images, videos, PSD, datasets repo-la direct commit panna repo size explode aagum. Git LFS (Large File Storage) actual file-ah separate store-la vekkum, repo-la pointer mattum irukkum.

terminal

```
git lfs install
git lfs track "*.psd"
git add .gitattributes design.psd
git commit -m "chore: track design files with LFS"
git lfs ls-files
```

COST NOTE

GitHub free plan-la LFS storage/bandwidth limit irukku. Already committed large file-ah LFS-ku move panna `git lfs migrate import --include="*.psd"` — history rewrite aagum, team coordination venum.

6.16 .gitattributes — Per-File Rules

Line endings, diff behavior, merge strategy, LFS tracking — file type-wise rules define panna. Windows + Mac team-la CRLF/LF issue solve pannum.

.gitattributes

```
* text=auto
*.sh text eol=lf
*.bat text eol=crlf
*.png filter=lfs diff=lfs merge=lfs -text
package-lock.json merge=binary
```

REAL-TIME IT COMPANY USAGE

Product teams-la `.gitattributes` mandatory: Windows developers CRLF commit panna CI Linux-la shell script fail aagum. `text=auto` + `eol=lf` standard. Design teams LFS use pandranga; legal/audit teams tag-based releases maintain pandranga.

Practice: stash create → apply → pop pannunga. `reset --soft` and `--mixed` try pannunga. `reflog` use panni oru commit recover pannunga. Annotated tag create panni remote-ku push pannunga.

CHAPTER 07 · TEAM COLLABORATION

Pull Requests & Collaboration

Fork, branch, PR, review, merge — company-la code team-ku pogum official route.

13 topics | Level: **Core team skill** | Time: **100 min** | Interviews: **Very high**

Company-la **main branch-la direct push illa**. Ellaa change-um branch → Pull Request → review → CI → merge. Indha flow theriyadhavan git therinjaalum team-la work panna mudiyadhu. Ithu dhaan chapter-oda main goal.

7.1 Fork — Your Own Copy

Fork = somebody else repo-oda complete copy, ungal account-la. Open source-la contribute panna first step. Original repo-ku **write access illa** — fork-la work panni PR vazhiya suggest pandrom. Company-la internal repos-ku fork usually vendaam; branch podhum.

terminal

```
# GitHub UI: repo → Fork → Create fork
git clone https://github.com/your-name/react.git
cd react
git remote add upstream https://github.com/facebook/react.git
git remote -v
```

ORIGIN VS UPSTREAM

`origin` = your fork (push access). `upstream` = original project (read-only). Fork-la commit pannunga, upstream-ku PR open pannunga.

7.2 Sync Your Fork

Upstream-la new commits vandhaal fork behind aagum. PR clean-a irukka regular-a sync pannunga. GitHub UI-la **Sync fork** button irukku; terminal-la:

terminal

```
git fetch upstream
git switch main
git merge upstream/main        # or git rebase upstream/main
git push origin main
```

COMMON MISTAKE

Fork-ah sync pannaama months-kaalam work pannina PR-la hundreds of unrelated commits varum. Feature branch create panna munnaadi always `git fetch upstream` pannunga.

7.3 Pull Request — What & Why

PR = "indha branch-la irukkura changes-ah main-ku merge pannunga" nu request. Code review, discussion, CI checks, approval, audit history — ellam PR-la nadakkum. PR = **quality gate**, not just merge button.

1. **Branch ready**
   Feature branch push pannunga: `git push -u origin feature/employee-search`.
2. **Open PR**
   GitHub → Pull requests → New pull request → base `main`, compare your branch.
3. **Describe**
   Clear title, what/why, screenshots, test steps, linked issue (`Closes #128`).
4. **Review + CI**
   Reviewer assign, checks pass aagum varaikkum wait, comments fix pannunga.
5. **Merge + cleanup**
   Approval + green checks aana merge pannunga, branch delete pannunga.

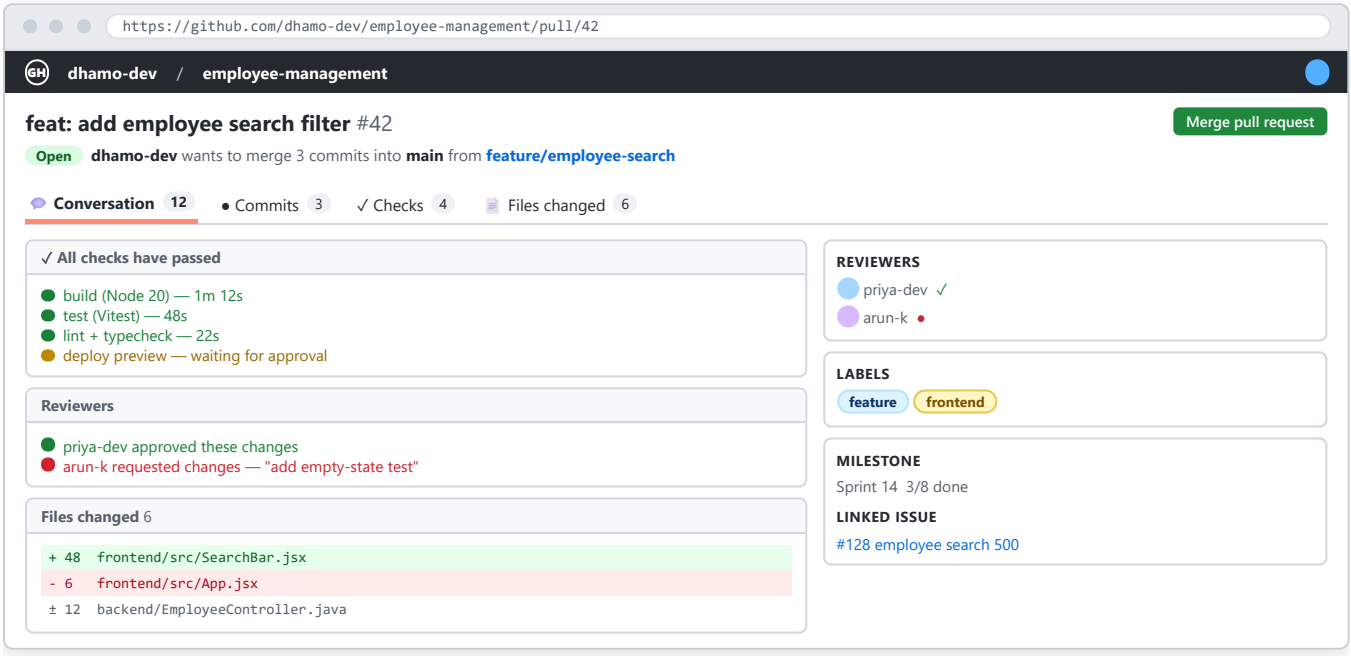


Fig 7.1 ILLUSTRATIVE MOCKUP **Pull request page.** Title, checks, reviewers, diff stats, labels, and linked issue — one screen-la full context.

7.4 PR Template

Repo-la `.github/PULL_REQUEST_TEMPLATE.md` vachaal, every new PR-la same checklist auto-fill aagum. Team consistency-ku idhu simple but powerful.

.github/PULL_REQUEST_TEMPLATE.md

```
## What changed
Employee search API + frontend filter bar.

## Why
Closes #128. Search was returning 500 for empty query.

## How tested
- npm test (12 passed)
- manual: empty, special chars, 500 rows

## Checklist
- [x] Tests added
- [x] No secrets committed
- [ ] Docs updated
```

PR TITLE MATTERS

`feat: add employee search` — good. `changes`, `fix`, `final code` — bad. Squash merge panna PR title dhaan main commit message aagum (Chapter 12 convention).

7.5 Code Review — Give & Receive

As author

Small PR (<400 lines) · self-review first · screenshots attach · reviewer question-ku reply · comment-ah resolve pannunga. "Nit:" prefix use pannina non-blocking suggestion nu clear aagum.

As reviewer

Logic, security, tests, naming, edge cases paarunga. Style-ah formatter handle pannum. **Approve** / **Request changes** / **Comment** — three options. Blocking comment-la reason + suggestion kudunga.

Review comment example — actionable

```
nit: indha magic number-ah constant-a extract pannalaam
suggestion: if (!query.trim()) return [];
question: empty query-ku 500 dhaan expected-a?
```

7.6 Suggested Changes

Reviewer line-level suggestion directly code-a edit panni propose pannalaam. Author **Commit suggestion** click pannina GitHub-la commit create aagum. Small fixes-ku romba fast; logic change-ku explanation venum.

GitHub review suggestion

```
- const results = employees.filter(e => e.name.includes(q));
+ const results = employees.filter(e =>
+   e.name.toLowerCase().includes(q.trim().toLowerCase())
+ );
```

7.7 Merge Strategies

Strategy	Result	Use when
Merge commit	All commits + merge commit preserved	Feature history valuable, release branch
Squash & merge	One clean commit on main	Messy WIP commits, most product teams
Rebase & merge	Linear history, individual commits kept	Every commit clean, trunk-based team

TEAM RULE

Repo settings-la **one strategy enforce** pannunga. Mix pannina history unpredictable aagum. Squash choose pannina PR title convention strict-a follow pannunga — adhu dhaan final commit message.

7.8 Draft Pull Request

Work in progress-ah early-a share panna draft PR open pannunga. CI run aagum, reviewers-ku context theriyum, but **merge button disabled**. Ready aana "Ready for review" click pannunga.

terminal

```
git push -u origin feature/employee-search
gh pr create --draft --title "feat: employee search" --body "WIP"
gh pr ready 42
```

WHY DRAFT?

Architecture decision early-a discuss pannalaam, duplicate work avoid aagum. Daily standup-la "draft PR #42 open" nu sollunga — transparency.

7.9 CODEOWNERS

File path-wise automatic reviewer assign panna. `.github/CODEOWNERS` create pannunga; branch protection-la "Require review from Code Owners" enable pannunga. Backend change-ku backend lead review mandatory aagum.

.github/CODEOWNERS

```
# default owners
*                   @dhamo-dev

# frontend
/frontend/          @priya-dev @arun-k

# backend + DB
/backend/           @dhamo-dev
/db/migrations/     @dba-team

# CI and release
/.github/workflows/ @devops-team
```

7.10 Branch Protection Rules

Main branch-ah accidental push/force push-la irundhu protect pannum. GitHub → Settings → Branches → Add branch protection rule. Company-la idhu **mandatory**.

Rule	Why
Require a pull request before merging	No direct push to main
Require approvals (1–2)	Four-eyes principle
Dismiss stale approvals	New commits-ku re-review
Require status checks	CI green aana dhaan merge
Require conversation resolution	All comments resolved
Require linear history	No merge commits (optional)
Block force pushes / deletions	History safe
Include administrators	Lead-um rule follow pannanum

MERGE QUEUE

PR traffic jasthi aana repo-la **merge queue** enable pannunga. GitHub PR-ah queue-la vekkum, latest main-la checks rerun panni pass aana mattum merge pannum — broken main avoid aagum.

7.11 Issues — Track Work

Bug, feature request, task — ellam issue. Every PR should link an issue: `Closes #128`. Merge aana issue automatic-a close aagum. Issue number-ah branch name-la use pannunga: `fix/128-empty-search`.

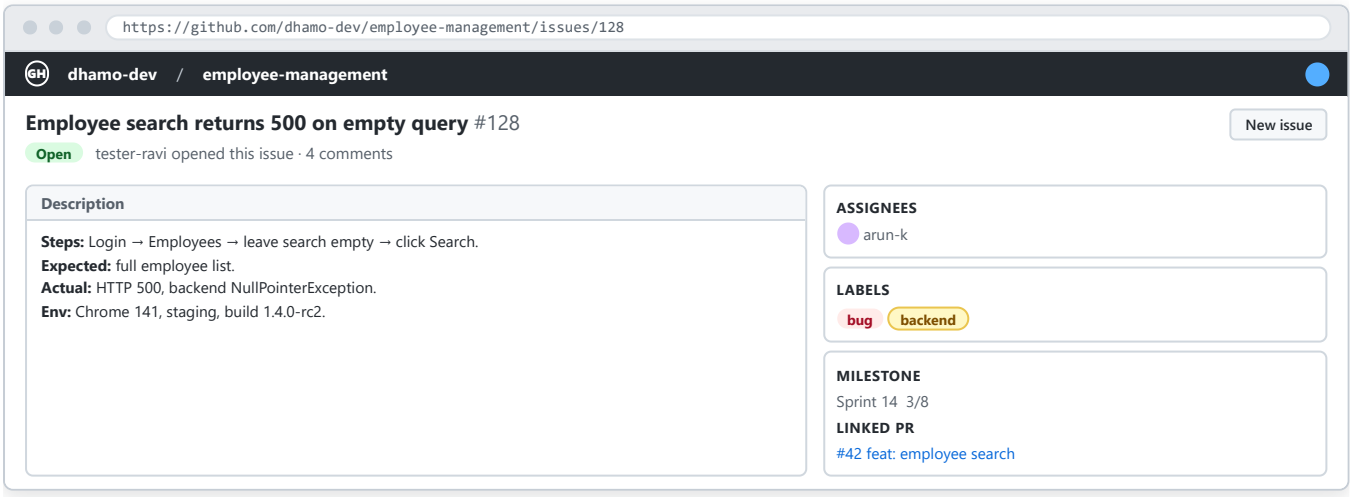


Fig 7.2 ILLUSTRATIVE MOCKUP **Issue detail page.** Steps, expected/actual, labels, assignee, milestone, and linked PR — no back-and-forth needed.

7.12 Labels & Milestones

Labels = category (bug, feature, frontend, priority). Milestone = deadline/release bucket (Sprint 14, v1.5). Rendum serndhu issue list-ah filter panna easy.

Label set

`bug` · `feature` · `docs` · `frontend` · `backend` · `priority:high` · `good first issue`.

Milestone rules

One issue = one milestone. Due date set pannunga. Sprint end-la open issues next sprint-ku move pannunga — carry-over visible aagum.

`.github/ISSUE_TEMPLATE/bug.yml`

```yaml
name: Bug report
description: Report a reproducible defect
title: "[Bug]: "
labels: ["bug", "triage"]
body:
  - type: textarea
    attributes:
      label: Steps to reproduce
      placeholder: 1. Login  2. Open Employees  3. Search empty
  - type: textarea
    attributes:
      label: Expected vs actual
  - type: input
    attributes:
      label: Environment (browser, build, env)
```

DISCUSSION → ISSUE

Open-ended idea discussion-la start aagum; actionable aana "Create issue" click panni convert pannunga. Discussion = conversation, issue = tracked work with owner and deadline.

7.13 Wiki & Discussions

Feature	Purpose	Example
Wiki	Long-form docs, versioned pages	Setup guide, architecture notes, runbook
Discussions	Open-ended Q&A, ideas, announcements	"Which state manager should we use?"
README	Quick start, first impression	Install, run, test, contribute
Issues	Actionable, closable work items	Bug #128, feature #131

REAL-TIME IT COMPANY USAGE

Typical flow: tester bug issue open pannuvaanga → TL priority label + sprint milestone set pannuvaanga → developer branch `fix/128-...` create panni PR open pannuvaanga → reviewer approve + CI green → squash merge → issue auto-close. Weekly-la stale PR/issue review pannuvanga.

Practice: oru repo fork pannunga, upstream add pannunga, feature branch-la change panni PR open pannunga. PR template + linked issue + draft PR + reviewer request ellam try pannunga.

CHAPTER 08 · CI/CD AUTOMATION

GitHub Actions & CI/CD

Push pannina automatic-a build, test, scan, deploy — manual work-ah machine-ku kudukkalam.

12 topics | Level: **Intermediate** | Time: **110 min** | Most asked: **DevOps rounds**

Company-la "code merge aanadhu" mattum podhaadhu — tests pass aagum, build success aagum, security scan clean-a irukkum, app deploy aagum. Indha ellam automatic-a nadakkanum. Adhu dhaan **CI/CD**; GitHub-la adhukku tool **GitHub Actions**.

8.1 CI/CD in One Minute

CI

Continuous Integration: every push-ku build + tests + lint. Bug early-a catch pannum.

CD (Delivery)

Artifact ready-a vekkum; manual approval-ku appuram deploy. Enterprise-la common.

CD (Deployment)

Pass aana udane production-ku automatic deploy. Startups/web apps-la common.

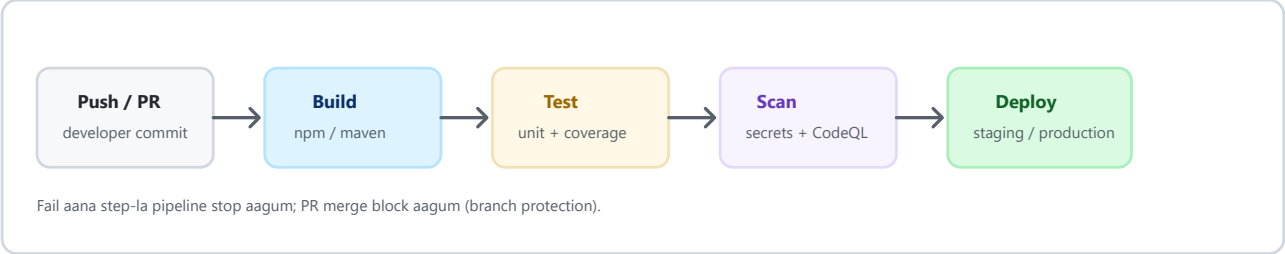


Fig 8.1 **Typical CI/CD pipeline.** Push triggers build → test → scan → deploy. Any red step stops the line.

8.2 Actions Building Blocks

Term	Meaning
Workflow	YAML file in `.github/workflows/`
Event	Trigger: push, pull_request, schedule, manual
Job	One unit of work; own runner; parallel by default
Step	Command or action inside a job; sequential
Action	Reusable package (checkout, setup-node, upload-artifact)
Runner	VM that executes the job (ubuntu, windows, mac)

8.3 Your First Workflow

Repo root-la `.github/workflows/ci.yml` create pannunga. Push pannina Actions tab-la run start aagum.

.github/workflows/ci.yml

```yaml
on:
  push: { branches: [main] }
  pull_request: { branches: [main] }
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: npm }
      - run: npm ci && npm test
```

READ IT LIKE A SENTENCE

"**On** push/PR to main, **run** build **on** ubuntu: checkout, Node 20, npm ci, test." YAML-la tab illa — 2 spaces.

8.4 Triggers — When Should It Run?

.github/workflows/ci.yml

```yaml
on:
  push:
    branches: [main, release/*]
  pull_request:
    branches: [main]
  workflow_dispatch:        # manual Run workflow button
  schedule:
    - cron: "0 2 * * *"     # daily 02:00 UTC
```

COMMON MISTAKE

`on: push` mattum vachaa PR checks enforce aagadhu — `pull_request` event venum. Paths filter: `paths: ["frontend/**"]`.

8.5 Jobs & Steps

.github/workflows/ci.yml

```yaml
jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci && npm run lint
  test:
    needs: lint
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci && npm test
```

needs

`needs: lint` → test job lint pass aana dhaan start aagum. Dependency graph.

Parallel vs serial

Jobs parallel; `needs` vachaa serial. Steps always serial.

8.6 Runners — Where It Runs

Runner	Spec (public repos)	Use for
`ubuntu-latest`	4 vCPU, 16 GB RAM, 14 GB SSD	Default; Docker, Node, Java
`windows-latest`	4 vCPU, 16 GB RAM	.NET, Windows installers
`macos-latest`	3 vCPU, 7 GB RAM	iOS, macOS builds
Self-hosted	Your own machine/VPC	Private network, GPU, compliance

MINUTES & COST

Private repos-ku free minutes limit irukku; macOS minutes 10× cost. Cache use pannunga (`cache: npm`). Self-hosted runner-la untrusted PR code run pannaadheenga — security risk.

8.7 Marketplace Actions

Common work-ku ready-made action use pannunga — own script ezhudha vendaam. Version pin pannunga (`@v4`), `@main` use pannaadheenga (supply-chain risk).

.github/workflows/ci.yml

```yaml
- uses: actions/checkout@v4
- uses: actions/setup-java@v4
  with:
    distribution: temurin
    java-version: 21
- uses: docker/build-push-action@v6
```

PIN BY SHA

High-security repos-la `uses: owner/action@<full-commit-sha>` pin pannunga. Tag mutable; SHA immutable. Dependabot GitHub Actions versions automatic-a update pannum.

8.8 Node CI — Complete Example

.github/workflows/node-ci.yml

```yaml
name: Node CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm
      - run: npm ci
      - run: npm run lint
      - run: npm test -- --coverage
      - uses: actions/upload-artifact@v4
        with:
          name: coverage
          path: coverage/
```

8.9 Matrix Builds

Same job-ah multiple versions/OS-la parallel-a run pannum. Library maintainers-ku must: Node 18/20/22, Ubuntu/Windows/Mac combinations.

.github/workflows/matrix.yml

```yaml
jobs:
  test:
    runs-on: ${{ matrix.os }}
    strategy:
      fail-fast: false
      matrix:
        os: [ubuntu-latest, windows-latest]
        node: [18, 20, 22]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node }}
      - run: npm ci && npm test
```

COST WARNING

2 OS × 3 versions = 6 jobs. `fail-fast: false` panna oru combo fail aana ellam run aagum (debug easy, cost jasthi). `include` / `exclude` use panni combinations control pannunga.

8.10 Artifacts — Keep Build Output

Job mudinjadhum runner delete aagum. Build file, test report, screenshot-ah **artifact**-a upload pannunga; 90 days default retention. Job-to-job file share-ku artifact use pannunga.

.github/workflows/build.yml

```yaml
- uses: actions/upload-artifact@v4
  with:
    name: app-build
    path: dist/
    retention-days: 14
- uses: actions/download-artifact@v4
  with:
    name: app-build
    path: dist/
```

8.11 Secrets & Variables

API key, DB password, deploy token-ah code-la ezhudhaadheenga. Repo → Settings → Secrets and variables → Actions-la add pannunga. Workflow-la `${{ secrets.NAME }}` use pannunga; logs-la automatic-a mask aagum.

Secret

Encrypted, masked. `secrets.DEPLOY_TOKEN`. Fork PR-ku secrets kidaadhu (safe).

Variable

Plain config: `vars.API_URL`. Environment-specific values-ku.

8.12 Deploy Workflow

.github/workflows/deploy.yml

```yaml
name: Deploy
on:
  push:
    tags: ["v*"]
permissions:
  contents: read
  pages: write
  id-token: write
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v4
      - run: npm ci && npm run build
      - uses: actions/deploy-pages@v4
```

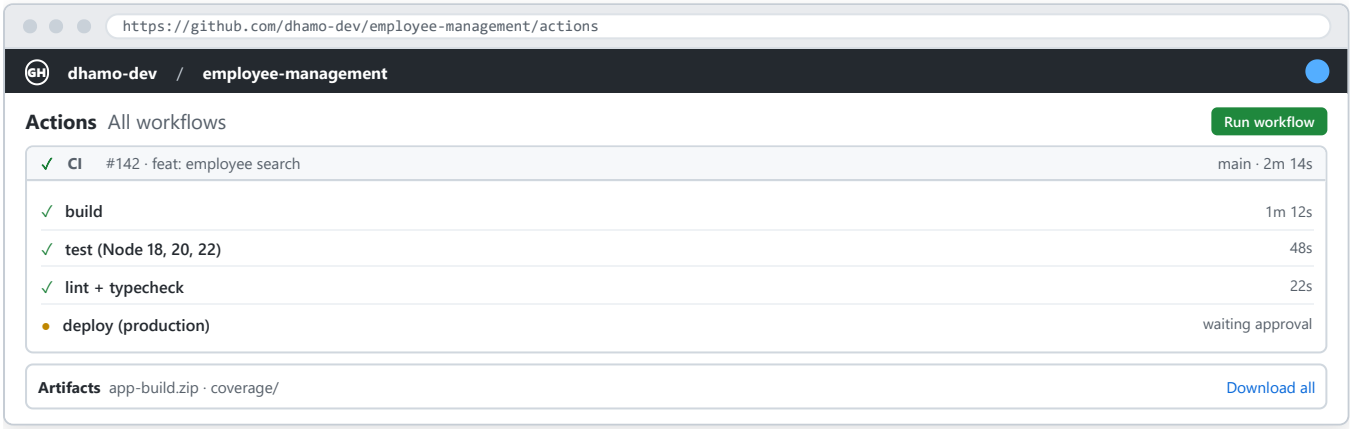


Fig 8.2 ILLUSTRATIVE MOCKUP **Actions run summary.** Jobs, durations, status, and artifacts. Production deploy approval-ku wait pannum.

REAL-TIME IT COMPANY USAGE

Product team-la PR open aana CI automatic-a run aagum; branch protection "status checks required" vachaa red CI merge-ah block pannum. Main merge aana staging deploy, tag push aana production. Secrets-ah environment-level vekkum — production token developers-ku theriyaadhu.

Practice: simple Node repo-la ci.yml create pannunga. Push panni Actions tab-la run paarunga. Oru test fail pannunga, red run observe pannunga, fix panni green aakkunga.

CHAPTER 09 · ACCOUNT & SUPPLY CHAIN

GitHub Security

Account protect, token manage, dependency scan, secret leak prevent — production-grade habits.

10 topics | Level: **Essential** | Time: **90 min** | Audit: **Mandatory**

Company repo-la oru leaked token = full breach. Indha chapter-la account-ah lock panna, machine-ah connect panna, dependencies-ah scan panna, secrets-ah block panna — ellam practical-a paakkom. Security = feature team-oda responsibility, "DevOps team work" illa.

9.1 Two-Factor Authentication (2FA)

Password mattum podhaadhu. 2FA = password + second proof (authenticator app, passkey, hardware key). GitHub-la **Settings → Password and authentication**; org owners enforce pannalaam. Recovery codes print panni safe-a vekkunga.

TOTP app
Microsoft/Google Authenticator. 6-digit code, 30s rotate. Easiest.

Passkey
Fingerprint/Face ID. Phishing-resistant, no code typing.

Security key
YubiKey USB/NFC. Highest assurance for admins.

RECOVERY CODES

Phone lose aana account lock. Recovery codes download panni password manager-la vekkunga. SMS 2FA weak — SIM swap attack irukku; TOTP/passkey preferred.

9.2 SSH Keys — Passwordless Identity

SSH key pair = private key (laptop-la, share pannaadhu) + public key (GitHub-la). HTTPS token-ah vida daily work-ku better. Oru key pair multiple GitHub accounts-ku use panna mudiyadhu — account-wise key venum.

terminal

```
ssh-keygen -t ed25519 -C "dhamo@example.com"
# Enter x3: default path, empty passphrase (or strong one)
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
cat ~/.ssh/id_ed25519.pub      # copy to GitHub → SSH keys
ssh -T git@github.com
```

MULTIPLE ACCOUNTS

`~/.ssh/config`-la Host alias create pannunga: `Host github-work` → `HostName github.com`, `IdentityFile ~/.ssh/id_work`. Remote: `git@github-work:org/repo.git`.

9.3 Personal Access Tokens (PAT)

HTTPS push, API call, CI script — password-ku badhil PAT. GitHub → Settings → Developer settings → Personal access tokens. **Fine-grained** token (repo-specific, read/write choose) classic-ah vida safe.

Type	Scope	Use
Fine-grained	One owner, selected repos, selected permissions	Recommended for scripts/CI
Classic	All repos, broad scopes (`repo`, `admin`)	Legacy only
GitHub App	Installation token, short-lived	Org-wide automation
OIDC	No stored secret; cloud trusts GitHub	CI → AWS/Azure deploy

NEVER COMMIT A TOKEN

Token repo-la commit aana GitHub push protection block pannum; but already pushed token **compromised**. Udane revoke → rotate → history clean (BFG/filter-repo). Expiry set pannunga: 30–90 days.

9.4 Secrets Management

Actions secret, environment secret, org secret — three levels. Production deploy token-ah **environment** secret-a vekkunga with required reviewers; developer-ku value theriyaadhu.

Do

Fine-grained token · short expiry · environment protection · OIDC for cloud · rotate on offboarding · audit access.

Don't

Token in code · `.env` commit · secrets in PR logs · personal token in shared CI · same token for all environments.

.github/workflows/deploy.yml

```yaml
- run: ./deploy.sh
  env:
    API_TOKEN: ${{ secrets.API_TOKEN }}
- uses: aws-actions/configure-aws-credentials@v4
  with:
    role-to-assume: arn:aws:iam::123:role/gh-deploy
    aws-region: ap-south-1
```

9.5 Dependabot — Dependency Updates

Old npm/Maven package-la CVE irundha Dependabot alert + automated PR open pannum. Weekly schedule, grouped updates, auto-merge for patch versions.

.github/dependabot.yml

```yaml
version: 2
updates:
  - package-ecosystem: npm
    directory: /frontend
    schedule: { interval: weekly }
    open-pull-requests-limit: 5
    groups:
      react: { patterns: ["react*"] }
  - package-ecosystem: maven
    directory: /backend
    schedule: { interval: weekly }
```

9.6 CodeQL & Code Scanning

CodeQL = semantic code analysis. SQL injection, XSS, path traversal, hardcoded credentials pola patterns kandupidikkum. **Security tab → Code scanning → Default setup** easiest; advanced workflow-la custom queries.

.github/workflows/codeql.yml

```yaml
name: CodeQL
on:
  push: { branches: [main] }
  pull_request: { branches: [main] }
  schedule: [{ cron: "0 3 * * 1" }]
jobs:
  analyze:
    runs-on: ubuntu-latest
    permissions: { security-events: write, contents: read }
    steps:
      - uses: actions/checkout@v4
      - uses: github/codeql-action/init@v3
        with: { languages: javascript, java }
      - uses: github/codeql-action/autobuild@v3
      - uses: github/codeql-action/analyze@v3
```

ALERT TRIAGE

Alert-ah **dismiss** panna reason kudunga (false positive / won't fix / used in tests). PR-la new alert vandha check fail aagum — merge block. Weekly security review-la open alerts zero-a vekkunga.

9.7 Secret Scanning & Push Protection

GitHub over 100 service patterns (AWS, Stripe, Slack) scan pannum. **Push protection** secret-ah push panna munnaadi block pannum. Custom pattern team-internal token-ku add pannalaam.

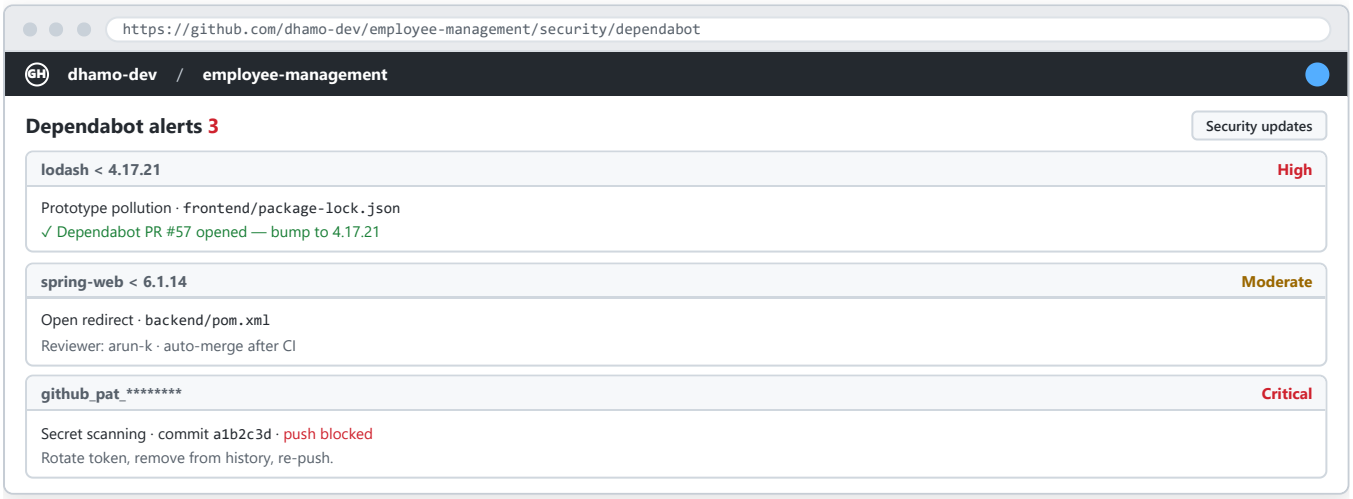


Fig 9.1 ILLUSTRATIVE MOCKUP **Security alerts.** Dependabot CVEs + secret scanning block — all in one Security tab.

9.8 Security Advisories

Your repo-la vulnerability irundha **privately** report pannalaam: Security → Advisories → New draft advisory. Maintainer fix panni patched version publish pannuvaanga; CVE request pannalaam. Open source-la responsible disclosure.

1. **Private report**
   Reporter-ku private discussion; public issue create pannaadheenga (exploit details leak aagum).
2. **Assess & fix**
   Affected versions list, patched version, severity (CVSS), CWE. Fix branch private-a maintain pannunga.
3. **Publish advisory**
   Patched release publish aana appuram advisory public pannunga; CVE ID request pannunga.
4. **Notify users**
   Release notes + advisory link + upgrade command. Dependabot alert automatic-a update aagum.

9.9 Signed Commits & Vigilant Mode

Commit really neenga dhaan pannitinga nu cryptographic proof. GPG or SSH key use panni sign pannalaam; GitHub "Verified" badge kaattum. Vigilant mode-la unsigned commits "Unverified" nu explicit-a kaattum.

terminal

```
git config --global user.signingkey ~/.ssh/id_ed25519.pub
git config --global commit.gpgsign true
git config --global gpg.format ssh
git commit -S -m "feat: signed commit"
git log --show-signature -1
```

BRANCH PROTECTION

"Require signed commits" enable pannina unsigned commit merge aagadhu. Regulated teams (finance/health) la mandatory. SSH signing GitHub 2022-la irundhu support pannum.

9.10 Least Privilege & Audit Log

Yaarukku enna access venum-nu mattum kudunga. Repo roles: **Read < Triage < Write < Maintain < Admin**. Contractors-ku read; CI bot-ku write specific repo. Org → Settings → Audit log-la every sensitive action irukkum.

Control	Action
Team-based access	Individual add pannaama team permissions
Audit log	Export to SIEM; alert on repo visibility change
IP allow list	Office/VPN-only access for enterprise
Offboarding	Remove org member, revoke PAT/SSH, rotate deploy keys

REAL-TIME IT COMPANY USAGE

Company-la 2FA org-wide enforced; PAT expiry 90 days; Dependabot + CodeQL + secret scanning ellam default on. Production deploy OIDC vazhiya; long-lived AWS keys illa. Quarterly access review + audit log export.

Practice: 2FA enable pannunga, SSH key add panni `ssh -T` test pannunga, fine-grained PAT create panni expiry set pannunga. Oru sample repo-la Dependabot config add panni alert/PR flow paarunga.

CHAPTER 10 · PLANNING & TRACKING

GitHub Projects

Issues-ah board-la drag panni, sprint plan panni, team progress-ah one screen-la paakkom.

9 topics | Level: **Team lead** | Time: **70 min** | Tool: **Projects v2**

Jira/Linear-ku alternative GitHub-native workflow. Issues + Projects + Milestones serndhu small/medium team-ku enough. Indha chapter-la board create panni, custom fields, automation, sprint report varaikkum paakkom.

10.1 Projects — What & Why

Project = issues/PRs-oda flexible view layer. Same data-ah board, table, roadmap-a paakalaam. Repo-la issue irukkum; project-la adhu **card** aagum. Multiple repos-oda issues-ah one project-la combine pannalaam.

Board
Kanban columns by status. Daily standup-ku best.

Table
Spreadsheet view: priority, size, assignee, sprint.

Roadmap
Timeline by start/target date. Leadership review.

10.2 Create & Link a Project

1. **Create**
   Profile/repo/org → Projects → New project → choose Board/Table/Roadmap template.

2. **Link repo**
   Project → ⋮ → Settings → Manage access → link repositories. Issues auto-add aagum.

3. **Add items**
   Existing issues add pannunga, or `+` panni new issue create pannunga. PR-um card aagum.

4. **Set status**
   Todo / In Progress / In Review / Done. Drag panna status field automatic-a update aagum.

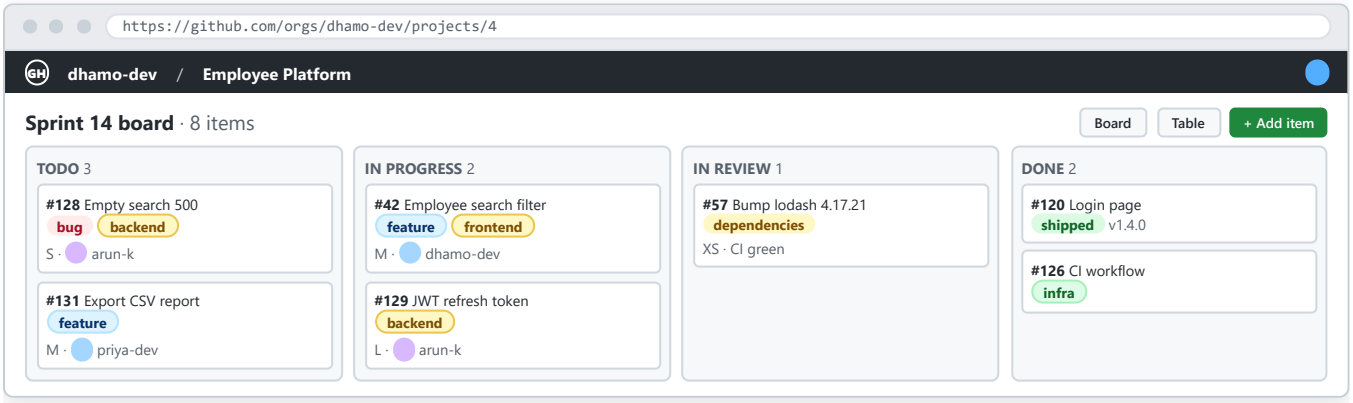


Fig 10.1 ILLUSTRATIVE MOCKUP **Sprint board.** Status columns, issue cards, labels, size, assignee — standup-ku single source of truth.

10.3 Board View — Daily Flow

Board = columns by a field (usually Status). WIP limit vachaa team-ah focus-a vekkum: In Progress 3-ku mela irundha new work start pannaadheenga. Card click pannina issue detail side panel-la open aagum.

STANDUP SCRIPT
Left to right paarunga: "Yesterday #42 move to review, today #131 start, blocker #128 backend fix wait." Board update pannaadha standup waste.

10.4 Table & Roadmap Views

Same project data, different lens. Table-la sort/filter/group pannalaam; roadmap-la start/target dates timeline-a paakalaam. View-ku per vachu team-ku share pannunga.

View	Best for	Key control
Board	Daily execution, WIP	Group by Status
Table	Backlog grooming, estimates	Sort by Priority, Size
Roadmap	Quarter planning, stakeholders	Start date + Target date

10.5 Custom Fields

Status, Assignee, Labels default. Team-ku venum: **Priority** (P0–P3), **Size** (XS–XL), **Sprint** (iteration), **Estimate** (number), **Target date**. Field add panna Table/Board/roadmap ellam use pannalaam.

Useful fields
Priority · Size · Sprint · Estimate · Target date · Team · Environment.

Anti-pattern
10+ fields, nobody fills. Start with 4; review monthly. Required fields enforce panna workflow-la validation step vekkunga.

10.6 Built-in Automation

Project → ⋮ → Workflows. Code ezhudhaama common automation enable pannalaam. Item added, closed, PR merged, review requested — ellam status/field update trigger aagum.

Workflow	Effect
Item added to project	Status = Todo
Issue closed	Status = Done
PR merged	Linked issue Status = Done
Auto-add to project	Repo issue/PR automatically add
Item reopened	Status = Todo

terminal

```
gh project list --owner dhamo-dev
gh project item-add 4 --owner dhamo-dev --url https://github.com/dhamo-dev/app/issues/131
gh project field-list 4 --owner dhamo-dev
```

10.7 Issue Templates & Forms

Bug/feature template standard-a vachaa triage fast. `.github/ISSUE_TEMPLATE/` la YAML forms create pannunga; required fields enforce pannalaam. Config-la blank issue disable pannunga.

.github/ISSUE_TEMPLATE/config.yml

```yaml
blank_issues_enabled: false
contact_links:
  - name: Security report
    url: https://github.com/dhamo-dev/app/security/advisories/new
    about: Do not open a public issue for vulnerabilities
```

TEMPLATE = TRIAGE SPEED

Bug form-la steps, expected, actual, environment required. Feature form-la problem, proposed solution, acceptance criteria. Template illaama "not working" nu issue varum — time waste.

10.8 Task Lists & Sub-Issues

Big issue-ah small tasks-a break pannunga. Task list markdown checklist; sub-issue parent-child progress automatic-a track aagum. PR-ku "Closes #128" link pannina task complete aagum.

Issue body — task list

```markdown
## Acceptance criteria
- [x] API accepts empty query
- [x] Returns full list, not 500
- [ ] Frontend empty state
- [ ] Tests cover empty + special chars
- [ ] Deployed to staging
```

SUB-ISSUES

Parent issue detail-la "Create sub-issue" click pannunga. Each sub-issue own assignee/status; parent progress bar auto update. Epic pattern-ku ideal.

10.9 Insights & Sprint Reporting

Project → Insights: burn-up, status distribution, velocity. Weekly review-ku 3 numbers: **throughput** (closed items), **WIP**, **carry-over**. Story points irundha velocity chart.

Burn-up

Completed vs total scope. Scope creep visible.

Throughput

Weekly closed issues. Trend matter, absolute illa.

Cycle time

Start → done days. Bottleneck kandupidikka.

REAL-TIME IT COMPANY USAGE

Team-la repo issues + org project board + sprint milestone. Daily standup board-la; weekly grooming table-la; monthly leadership roadmap-la. Automation status update pannum, manual status mail illa.

Practice: oru project create pannunga, repo link pannunga, 5 issues add pannunga. Board/table/roadmap views create panni, Priority + Size fields add panni, "Issue closed → Done" workflow enable pannunga.

CHAPTER 11 · FREE HOSTING

GitHub Pages

Portfolio, docs, landing page — repo-la irundhu free static website publish pannalam.

7 topics | Level: **Beginner+** | Time: **60 min** | Cost: **Free**

Recruiter-ku live portfolio link kudukkanum? Project docs professional-a host pannanum? GitHub Pages = static files-ah CDN-la free-a serve pannum. Public repos-ku free; private repos-ku paid plan (org policy paarunga).

11.1 What Are GitHub Pages?

Static site hosting: HTML, CSS, JS, images. Server-side code (Node/Java/PHP) run aagadhu — adhukku Render/Railway/Fly.io venum. Build step (React/Vite/Jekyll) use pannalaam; output static files mattum serve aagum.

Project site
`user.github.io/repo` — repo-ku subpath.

User site
`user.github.io` — repo name `user.github.io`.

Custom domain
`www.example.com` — CNAME + DNS.

11.2 Deploy from a Branch

1. **Push static files**
   `index.html` repo root-la or `/docs` folder-la vekkunga.
2. **Open settings**
   Repo → Settings → Pages → Build and deployment.
3. **Select source**
   Deploy from a branch → branch `main` → folder `/ (root)` or `/docs` → Save.
4. **Wait & visit**
   1–2 minutes-la URL live. Actions tab-la "pages build and deployment" run paarunga.

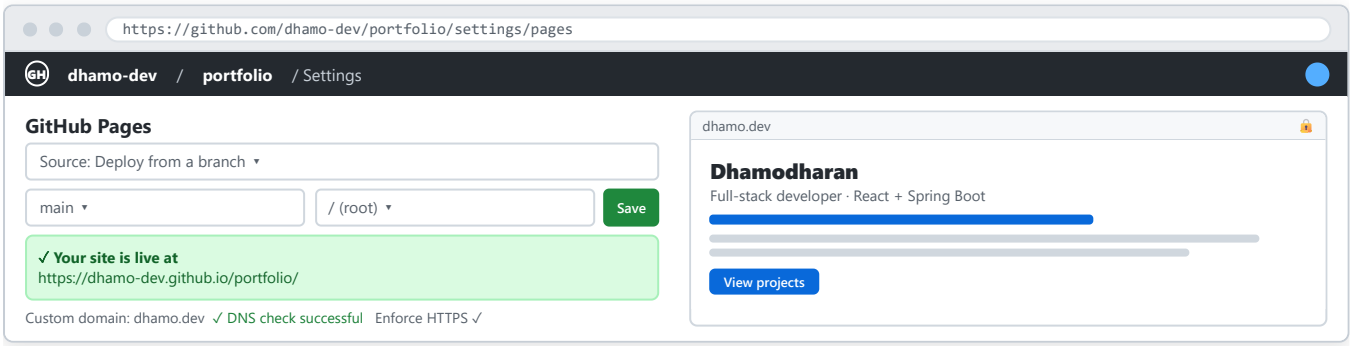


Fig 11.1 ILLUSTRATIVE MOCKUP **Pages settings + live site.** Branch source, custom domain, HTTPS, and the deployed portfolio.

11.3 Deploy with GitHub Actions

Build step venum-na (Vite, Next export, Jekyll) Actions workflow use pannunga. Settings → Pages → Source = **GitHub Actions**. Workflow build panni artifact upload pannum.

.github/workflows/pages.yml

```yaml
name: Deploy Pages
on:
  push: { branches: [main] }
permissions:
  contents: read
  pages: write
  id-token: write
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: npm }
      - run: npm ci && npm run build
      - uses: actions/configure-pages@v5
      - uses: actions/upload-pages-artifact@v3
        with: { path: dist }
  deploy:
    needs: build
    runs-on: ubuntu-latest
    environment: github-pages
    steps:
      - uses: actions/deploy-pages@v4
```

SPA ROUTING

React Router BrowserRouter deep link (e.g. `/about`) refresh-la 404 aagum. `dist/404.html` copy of index.html vekkunga, or HashRouter use pannunga.

11.4 Custom Domain

1. **DNS records**
   Apex domain `example.com`: A records to GitHub Pages IPs. `www`: CNAME to `user.github.io`.
2. **Add in GitHub**
   Settings → Pages → Custom domain → enter domain → Save. DNS check pass aagum varaikkum wait.
3. **CNAME file**
   Branch deploy-la repo root-la `CNAME` file (domain name only) commit pannunga — custom domain retain aagum.
4. **Verify**
   `dig example.com` / `nslookup` check pannunga. Propagation 24h varaikkum aagalaam.

11.5 HTTPS & Security

Custom domain add pannina GitHub automatic-a Let's Encrypt certificate issue pannum (few minutes). Settings-la **Enforce HTTPS** checkbox enable pannunga — HTTP request HTTPS-ku redirect aagum. Mixed content (http:// image/script) irundha padhu.

11.6 Jekyll Static Site

Jekyll = Ruby static site generator. Markdown + Liquid templates → HTML. GitHub Pages default-a Jekyll build pannum (branch source); no custom workflow needed. Blog/docs-ku ideal.

_config.yml

```
title: Dhamo Dev Notes
description: React, Spring Boot, Git
theme: minima
baseurl: "/notes"
url: "https://dhamo-dev.github.io"
markdown: kramdown
plugins:
  - jekyll-feed
  - jekyll-seo-tag
```

NO RUBY LOCALLY?

GitHub-la edit panni commit pannalaam — Pages build pannum. Local preview-ku `bundle exec jekyll serve` (Ruby + Bundler venum). Theme files override panna `_layouts/` folder.

11.7 React SPA on Pages

Vite React app-ah static-a build panni host pannalaam. `vite.config.js`-la `base` set pannunga (project site subpath), build output `dist/` push/upload pannunga.

vite.config.js

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()],
  base: "/employee-management/",   // repo name
});
```

ENVIRONMENT NOTE

Pages = static only. API calls-ku external backend (Render/Railway) venum. CORS allow pannunga; API URL-ah build-time env variable-la vekkunga (`VITE_API_URL`). Secrets frontend bundle-la vekkaadheenga — ellam public.

REAL-TIME IT COMPANY USAGE

Teams internal docs-ku Pages use pandranga: architecture decision records, API docs, onboarding guide. Portfolio/demo sites-ku free hosting. Production customer app-ku Pages illa — auth, SSR, API venum; adhukku cloud platform.

Practice: repo-la simple index.html push panni Pages enable pannunga. Custom domain (or username.github.io) set panni HTTPS enforce pannunga. Vite React build-ah `dist/` branch-la deploy pannunga.

CHAPTER 12 · CAREER & CRAFT

Professional GitHub

Profile, commits, docs, releases, CLI — interviewer-ku neenga serious developer nu kaattum signals.

12 topics | Level: **Career** | Time: **80 min** | Impact: **High**

Code work aagum; but professional-a irukkanum. Clear commit messages, semantic versions, changelog, docs, clean repo — indha habits team-la trust build pannum. Recruiter ungal GitHub profile-ah 30 seconds-la judge pannuvaanga.

12.1 Profile README

Repo name ungal username-a irundha (`dhamo-dev/dhamo-dev`) adhu profile page-la render aagum. Role, stack, current projects, contact — one screen summary. Pinned repos 6 varaikkum select pannunga.

dhamo-dev/README.md

```
### Hi, I'm Dhamodharan

Full-stack developer — React, Spring Boot, PostgreSQL.

- Currently building: employee-management (React + Spring Boot)
- Learning: GitHub Actions, Kubernetes
- Portfolio: https://dhamo.dev
- Contact: dhamo@example.com

**Stack:** Java 21 · Spring Boot · React · TypeScript · Docker
```

RECRUITER CHECKLIST

Profile photo · one-line bio · pinned 3–6 real projects · each repo README with screenshot + run steps · live demo link. Tutorial clones-ah pin pannaadheenga.

12.2 Contribution Graph

Daily commits-ah green squares-a kaattum. Consistency signal, but **meaningless commits-ah padhu** — recruiters commit count paakka maataanga; quality, tests, docs paapaanga. Private contributions count enable pannalaam.

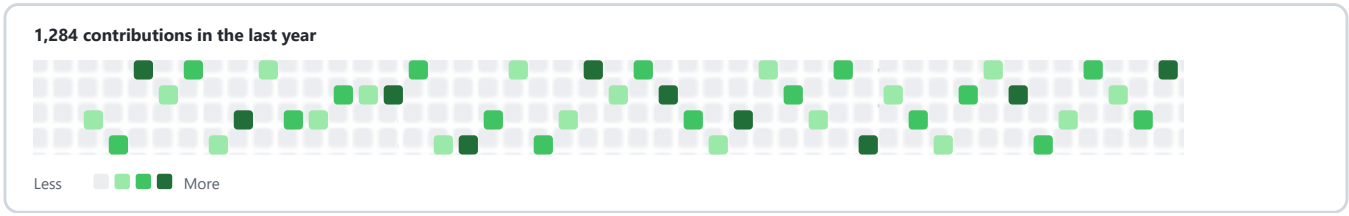


Fig 12.1 ILLUSTRATIVE MOCKUP **Profile contribution graph.** Daily consistency; empty weeks normal — vacation, focus work.

12.3 Open Source Contribution

1. **Start small**
   `good first issue` / `documentation` label filter pannunga.
2. **Read CONTRIBUTING**
   Setup steps, branch naming, test command, DCO/CLA requirements.
3. **Claim & discuss**
   Issue-la "I'd like to work on this" comment pannunga; approach confirm pannunga.
4. **PR with tests**
   Small focused PR, tests pass, maintainer review-ku respond pannunga. Patience — days/weeks aagalaam.

12.4 Commit Message Convention

Conventional Commits = `type(scope): summary`. Changelog generation, semantic release, readable history — ellam idhu-la irundhu varum. Subject 50 chars, imperative mood ("add" not "added"), body-la why explain pannunga.

terminal

```
feat(search): add empty-query handling
fix(auth): refresh token before expiry
docs(readme): add local setup steps
refactor(api): extract employee mapper
test(search): cover special characters
```

Type	Meaning
`feat`	New user-facing feature (MINOR)
`fix`	Bug fix (PATCH)
`docs`	Documentation only
`refactor`	No behavior change
`test`	Tests added/fixed

BREAKING CHANGE

`feat(api)!: remove legacy endpoint` or footer `BREAKING CHANGE: ...` → MAJOR version bump. CI-la commitlint hook + PR title check vachaa convention enforce aagum.

12.5 Semantic Versioning

`MAJOR.MINOR.PATCH` — library/API versioning standard. Breaking change → MAJOR; backward-compatible feature → MINOR; bug fix → PATCH. `0.y.z` = initial development, API unstable.

Change	Version	Example
Breaking API change	2.3.1 → **3.0.0**	Remove field, change response shape
New feature	2.3.1 → **2.4.0**	Add filter endpoint
Bug fix	2.3.1 → **2.3.2**	Fix null pointer
Pre-release	**3.0.0-rc.1**	Release candidate

12.6 Changelog

`CHANGELOG.md` = humans-ku readable release history. "Keep a Changelog" format: Added, Changed, Deprecated, Removed, Fixed, Security. Tag date kudunga; latest top-la.

CHANGELOG.md

```
## [1.4.0] - 2026-11-09
### Added
- Employee search filter with empty-query handling (#128)
- CSV export endpoint (#131)

### Fixed
- JWT refresh race condition on tab switch (#126)

### Security
- Bump lodash to 4.17.21 (CVE-2021-23337)
```

12.7 Review Etiquette

Comment style

Code-ah critique pannunga, person-ah illa. "This query runs per row" > "you wrote bad code". Suggestion + reason kudunga. `nit:` = non-blocking.

Response style

Agree / explain / pushback respectfully. "Good catch, fixed in 3f2a1b" or "Keeping it because...". Resolve button use pannunga.

24-HOUR RULE

PR open panni 24 hours-la first review venum; reviewer-ku 2 working days. Blocked-a irundha standup-la raise pannunga — silent PR = team bottleneck.

12.8 Documentation & README

README = project front door. New developer 10 minutes-la run panna venum. Screenshot, prerequisites, setup commands, env variables, test command, architecture one-liner, contribution link.

README.md

```
# Employee Management

React + Spring Boot CRUD app with JWT login.

## Quick start
1. `docker compose up -d db`
2. `cd backend && ./mvnw spring-boot:run`
3. `cd frontend && npm ci && npm run dev`
4. Open http://localhost:5173

## Environment
Copy `.env.example` to `.env` and fill DB_URL, JWT_SECRET.

## Tests
`./mvnw test` · `npm test`
```

12.9 .gitignore

Generated files, secrets, IDE files commit aagakoodadhu. Language-specific template use pannunga (github/gitignore). Already tracked file ignore panna `git rm --cached file` venum.

.gitignore

```
node_modules/
dist/
target/
.env
.env.*
!.env.example
*.log
.idea/
.vscode/
.DS_Store
coverage/
```

12.10 Repository Structure

Predictable structure = onboarding fast. Monorepo-la `frontend/` + `backend/` + `infra/` separate pannunga. Root-la README, LICENSE, .gitignore, CONTRIBUTING, CHANGELOG.

repository tree

```
employee-management/
├── frontend/          React + Vite
├── backend/           Spring Boot
├── infra/             Docker, compose, k8s
├── docs/              ADRs, API docs
├── .github/           workflows, templates, CODEOWNERS
├── README.md
├── CHANGELOG.md
├── CONTRIBUTING.md
└── LICENSE
```

NAMING RULES

lowercase-with-hyphens for files/folders; `PascalCase` components; `camelCase` Java methods? No — Java `camelCase`, classes `PascalCase`, constants `UPPER_SNAKE`. Team convention document pannunga.

12.11 Gists

Small snippet/note share pannalaam — full repo vendaam. Secret gist = unlisted (not private!), public gist = searchable. Code snippet, config sample, interview notes-ku useful; project code-ku repo use pannunga.

Use	Tool
One-file snippet	Gist
Full project	Repository
Long docs	Repo + Pages / Wiki
Quick question	Discussion / Stack Overflow

12.12 GitHub CLI (gh)

Terminal-la repo, PR, issue, Actions, release manage pannalaam. Scripting-ku powerful; browser context switch avoid aagum. Install: `winget install GitHub.cli`, then `gh auth login`.

terminal

```
gh repo clone dhamo-dev/employee-management
gh pr create --fill
gh pr checks 42
gh pr review 42 --approve
gh issue list --label bug
gh run list --limit 5
gh release create v1.4.0 --generate-notes
```

REAL-TIME IT COMPANY USAGE

Teams commitlint + semantic-release use pandranga: merge commit message-la irundhu version + changelog automatic. Docs repo-la ADR (Architecture Decision Record) maintain pandranga. CLI scripts CI/CD-la common.

Practice: profile README create pannunga, 3 repos pin pannunga. Oru PR-la Conventional Commit title use pannunga. CHANGELOG entry add pannunga. `gh pr create --fill` try pannunga.

CHAPTER 13 · REAL PROJECT SIMULATION

Real Team Workflow

React + Spring Boot team-la sprint start muthal production release varaikkum — end-to-end.

12 topics | Team: **6 members** | Sprint: **2 weeks** | Level: **Job-ready**

Ithu dhaan final integration chapter. Ellaa command-um theriyum, but real team-la **yaar enna pannuvaanga, eppu pannuvaanga** nu theriyanum. Employee management app (React frontend + Spring Boot backend) team example eduthukkalam.

13.1 Team Roles & Ownership

Role	Member	Owns
Tech Lead	Karthik	Architecture, PR final approval, release, production hotfix
Backend	Arun	Employee API, JWT, DB migrations, backend tests
Backend	Meera	Search/report API, validation, performance
Frontend	Priya	React components, routing, state, accessibility
Frontend	Dhamo	Search UI, API integration, E2E tests
Tester	Ravi	Test cases, bug reports, staging verification, release sign-off

13.2 Sprint Flow — Two Weeks

1. **Planning (Day 1)**
   TL backlog groom pannuvaanga; team estimate pannum; issues sprint milestone-ku move aagum; owners assign aagum.

2. **Development (Day 2–8)**
   Feature branch → small commits → draft PR early → daily push → CI green.

3. **Review & QA (Day 8–9)**
   Reviewer approval, tester staging-la verify, bug issues create, fixes same branch-la.

4. **Release (Day 10)**
   TL tag create, production deploy, smoke test, release notes, retrospective.

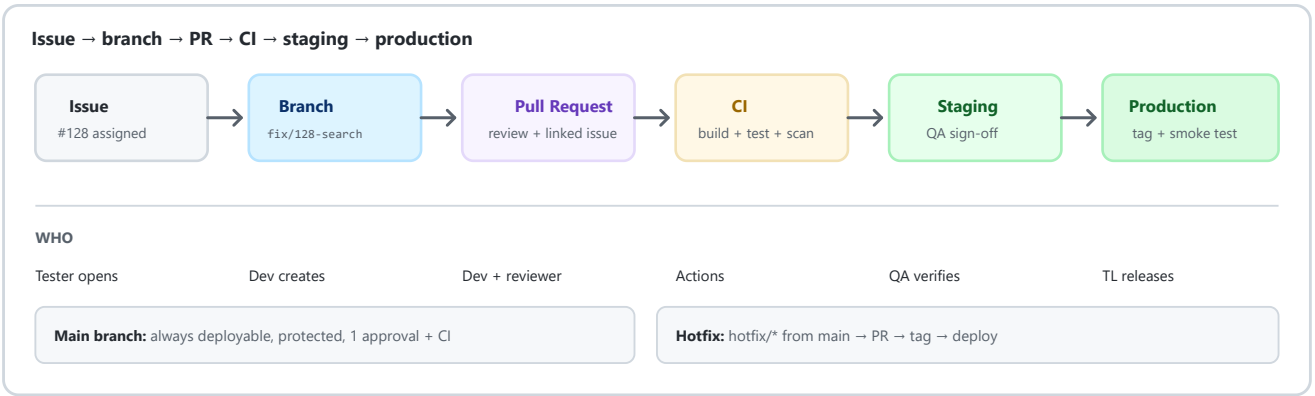


Fig 13.1 **Team delivery workflow.** Every change same path-la pogum — no shortcuts, no direct main commits.

13.3 Issue Assignment

Tester bug open pannuvaanga; TL triage panni priority, owner, sprint set pannuvaanga. Developer "I'll take this" comment panni status In Progress-ku move pannunga. Duplicate work avoid aagum.

Issue #128 — assignment

```
Title: Employee search returns 500 on empty query
Assignee: arun-k
Labels: bug, backend, priority:high
Milestone: Sprint 14
Branch: fix/128-empty-search
```

13.4 Branch Naming Standard

Pattern	Example
`feature/<issue>-slug`	`feature/131-csv-export`
`fix/<issue>-slug`	`fix/128-empty-search`
`hotfix/<version>`	`hotfix/1.4.1-payment`
`chore/<slug>`	`chore/update-spring-boot`

terminal

```
git switch main
git pull --ff-only
git switch -c fix/128-empty-search
```

13.5 React Frontend Flow

1. **Start from main**
   Branch create, `npm ci`, `npm run dev`. Component + test file create pannunga.
2. **API contract**
   Backend swagger/openapi paarunga; mock data-la start pannunga. `VITE_API_URL` env use pannunga.
3. **Test & lint**
   `npm test`, `npm run lint`. Empty/loading/error states cover pannunga.
4. **Draft PR early**
   Screenshot attach pannunga; reviewer-ku UI context kudunga. Tester preview URL-la verify pannuvaanga.

13.6 Spring Boot Backend Flow

1. **Branch & DB**
   Branch create, local Postgres start, migration script add pannunga (Flyway/Liquibase).
2. **Code layers**
   Controller → Service → Repository. Validation + exception handler add pannunga.
3. **Tests**
   Unit + integration tests: `./mvnw test`. Empty query, SQL injection, auth cases cover pannunga.
4. **PR + API doc**
   Swagger update pannunga. Backend PR-la migration order + rollback note kudunga.

13.7 PR Flow — Feature to Merge

1. **Push branch**
   `git push -u origin fix/128-empty-search` → GitHub "Compare & pull request" click pannunga.
2. **Fill template**
   What/why/test steps; `Closes #128`; screenshots; reviewer + labels + milestone set pannunga.
3. **CI green**
   Build, unit, lint, CodeQL pass aagum varaikkum wait. Fail aana same branch-la fix + push.
4. **Review**
   Reviewer comments address pannunga; re-request review. Approve aana dhaan merge button enable.
5. **Squash & merge**
   PR title clean-a vachu squash merge. Branch delete. Issue auto-close. Staging deploy trigger.

terminal

```
git add src/EmployeeService.java
git commit -m "fix(search): guard empty query"
git push
gh pr create --fill --reviewer karthik-tl
gh pr checks 42
```

13.8 Review & Fix Cycle

Reviewer finds
Null check missing · test case missing · N+1 query · hardcoded URL · sensitive log line.

Author fixes
Same branch-la new commit · reply "fixed in abc123" · resolve conversation · re-request review.

DON'T FORCE-PUSH DURING REVIEW
Reviewer comments-oda commit context lose aagum. New commits add pannunga; merge-la squash pannina history clean-a irukkum. Force push only your own unreviewed branch.

13.9 Conflict Handling in Team

Main fast-moving; your branch 3 days old aana conflict varum. **Daily pull/rebase** pannunga. Conflict vandha panic aagaadheenga — both changes understand panni combine pannunga.

terminal

```
git fetch origin
git rebase origin/main
# resolve conflicts in VS Code
git add <resolved-files>
git rebase --continue
git push --force-with-lease
```

TEAM PROTOCOL
Conflict resolution-la unsure-a irundha original author-ku ping pannunga. "Ours/theirs" blindly select pannaadheenga. Resolve panna appuram tests local-la run pannunga.

13.10 CI Tests — Merge Gate

PR-la unit + integration + lint + security scan run aagum. Branch protection-la indha checks required-a vekkunga. Red CI merge-ah block pannum — "later fix" culture avoid aagum.

.github/workflows/pr.yml

```yaml
name: PR Checks
on: { pull_request: { branches: [main] } }
jobs:
  backend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: 21 }
      - run: cd backend && ./mvnw verify
  frontend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: npm }
      - run: cd frontend && npm ci && npm test -- --run
```

13.11 Deploy Pipeline

1. **Merge to main**
   Staging auto-deploy aagum. Tester smoke test + regression run pannuvaanga.

2. **Tag release**
   TL `v1.4.0` tag push pannuvaanga. Production deploy workflow trigger aagum.

3. **Approval gate**
   GitHub Environment "production" required reviewers; TL approve pannuvaanga.

4. **Smoke test**
   Health endpoint, login, search, one critical flow check. Fail aana rollback.

ROLLBACK READY
Deploy-ku munnaadi previous tag + DB migration rollback plan ready-a vekkunga. Feature flag irundha deploy-ah off panni control pannalaam.

13.12 Release & Hotfix

terminal

```bash
# scheduled release
git switch main && git pull
git tag -a v1.4.0 -m "Sprint 14 release"
git push origin v1.4.0

# production hotfix
git switch -c hotfix/1.4.1-payment main
# fix + test + PR + merge
git tag -a v1.4.1 -m "Fix payment timeout"
git push origin v1.4.1
```

13.13 Sprint Simulation — Day by Day

Day	What happens	Git/GitHub action
1	Planning, estimate, assign	Issues → Sprint 14 milestone, owners
2	Arun backend branch start	`fix/128-empty-search`, draft PR
3	Priya search UI branch	`feature/131-csv-export`, mock API
5	Mid-sprint sync	Board update, blockers, CI status
7	Backend PR review	2 comments, fixes, approval
8	Squash merge + staging	Merge, branch delete, auto deploy
9	QA regression + sign-off	Bug issues, fixes, re-test
10	Release + retro	Tag v1.4.0, release notes, deploy

13.14 Definition of Done

Code
Reviewed + approved · tests added · lint clean · no secrets · migration tested · docs updated.

Delivery
CI green · staging verified · release note · monitoring dashboard · rollback plan.

TEAM ANTI-PATTERNS
Direct push to main · PR without linked issue · "LGTM" review without reading · red CI merge · force push shared branch · secrets in code · 2-week-old branch · release without tag.

13.15 Metrics That Matter

Lead time
Issue start → production. Target: days, not weeks.

Review time
PR open → first review. Target: < 24h.

Change failure
Deploy fail %. Target: < 15%, rollback fast.

REAL-TIME IT COMPANY USAGE
Indha flow dhaan product companies-la standard: protected main, short branches, PR template + CODEOWNERS, CI required, staging auto-deploy, production approval + tag, weekly release. Team size 5–8-ku GitHub-native workflow enough; enterprise-la Jira/Azure DevOps layer mela irukkum, but Git flow same.

Final practice: 6-member team simulate pannunga. Oru issue create → assign → branch → PR → review → CI → merge → tag → release notes. Full cycle oru repo-la end-to-end pannunga.

APPENDIX A · QUICK REFERENCE

Command Cheat Sheet

Daily commands, one page-la. Bookmark pannunga, terminal side-la vekkunga.

2 pages | Level: All | Use: Daily

A.1 Setup & Configuration

terminal

```
git config --global user.name "Dhamodharan"
git config --global user.email "dhamo@example.com"
git config --global init.defaultBranch main
git config --global pull.rebase true
git config --list --show-origin
git init
git clone https://github.com/user/repo.git
```

A.2 Daily Workflow

Command	What it does
`git status`	Working tree + staging state
`git diff` / `git diff --staged`	Unstaged / staged changes
`git add <file>` / `git add -p`	Stage file / stage hunks
`git commit -m "feat: ..."`	Commit staged changes
`git commit --amend --no-edit`	Add files to last local commit
`git log --oneline --graph --all`	Visual history
`git blame <file>`	Line-wise author + commit

A.3 Branch & Merge

terminal

```
git branch -a                # list local + remote
git switch -c feature/login  # create + switch
git merge feature/login      # merge into current
git merge --no-ff feature/x  # always merge commit
git merge --abort            # cancel conflicted merge
git branch -d feature/login  # delete merged branch
git push origin --delete feature/login
```

A.4 Remote & Sync

Command	What it does
`git remote -v`	Show remote URLs
`git remote add upstream <url>`	Add original repo (fork)
`git fetch origin`	Download refs, don't merge
`git pull --ff-only`	Fetch + fast-forward only
`git push -u origin main`	Push + set upstream
`git push --force-with-lease`	Safe force push (own branch)

A.5 Undo & Recovery

Command	Use when
`git restore file`	Discard unstaged file changes
`git restore --staged file`	Unstage, keep working change
`git reset --soft HEAD~1`	Undo commit, keep staged
`git reset --mixed HEAD~1`	Undo commit + unstage
`git reset --hard HEAD~1`	Discard commit + changes
`git revert <hash>`	Undo pushed commit safely
`git reflog`	Find lost commits
`git stash push -m "wip"`	Park unfinished work
`git stash pop`	Restore + remove stash

A.6 Advanced

terminal

```
git rebase -i HEAD~3           # squash/reword/drop
git rebase origin/main         # update feature base
git cherry-pick 8f2c1ab        # copy one commit
git tag -a v1.0.0 -m "release" # annotated tag
git push origin v1.0.0
git worktree add ../hotfix hotfix/1.0.1
git bisect start               # binary search bug
git lfs track "*.psd"
```

A.7 GitHub & CLI

Command / action	Purpose
`gh auth login`	Authenticate CLI
`gh repo clone owner/repo`	Clone repo
`gh pr create --fill`	PR from current branch
`gh pr checks`	CI status
`gh pr review --approve`	Approve PR
`gh issue list --label bug`	Filter issues
`gh run list`	Recent workflow runs
`gh release create v1.4.0`	Create release

A.8 Danger Commands

THINK TWICE

`reset --hard` · `push --force` · `clean -fdx` · `branch -D` · `stash clear` · `filter-repo` · `rm -rf .git`. Shared branch-la force push never; uncommitted work first `git stash`.

APPENDIX B · TROUBLESHOOTING

Common Errors & Fixes

Error message paarunga, panic aagaadhinga. Ithu 14 repeat offenders — cause and exact fix.

14 errors | Level: **All** | Use: **When stuck**

Error message	Why it happens	Exact fix
`fatal: not a git repository`	Wrong folder, or repo not initialised	`cd` into project, then `git init`
`Permission denied (publickey)`	SSH key missing or not added to GitHub	`ssh -T git@github.com`, add `~/.ssh/id_ed25519.pub`
`Authentication failed`	Old password cached; GitHub needs a token	Clear Windows Credential Manager, use a PAT
`src refspec main does not match any`	No commit yet, or branch named `master`	`git add -A` + `git commit`, then `git branch -M main`
`Updates were rejected … fetch first`	Remote has commits your clone lacks	`git pull --rebase`, resolve, then `git push`
`CONFLICT (content): Merge conflict`	Same lines changed on both branches	Edit markers, `git add <file>`, `git commit`
`You have not concluded your merge`	A merge stopped midway (`MERGE_HEAD`)	Finish it, or `git merge --abort`
`Your local changes would be overwritten`	Dirty working tree blocks switch/pull	Commit, or `git stash` → switch → `git stash pop`
`You are in 'detached HEAD' state`	Checked out a commit or tag, not a branch	`git switch -c my-branch` to keep work
`fatal: bad object / unknown revision`	Typo, or commit never fetched	`git fetch --all`, verify with `git log --all --oneline`
`error: remote origin already exists`	Remote added twice	`git remote set-url origin <url>`
`LF will be replaced by CRLF`	Windows line endings (warning, not fatal)	`git config core.autocrlf true` + `.gitattributes`
`GH013: protected branch`	Direct push blocked by ruleset	Push a branch and open a PR instead
`refusing to merge unrelated histories`	Two repos with separate first commits	`git pull origin main --allow-unrelated-histories`

1 First-aid commands

`git status` → what state am I in?
`git log --oneline -5` → what did I just do?
`git remote -v` → am I pushing to the right place?
`git reflog` → where did that commit go?

2 Read the message properly

fatal = Git stopped. **error** = action failed. **warning** = it continued. **hint** = Git is telling you the next command — read the last three lines first.

NEVER DO THIS WHILE DEBUGGING

Error varumbothu `git reset --hard` or `rm -rf .git` adikkadhinga. First `git stash` or a backup branch (`git branch backup-today`) create pannunga — then experiment freely. Shared branch-la force push always no.

APPENDIX C · INTERVIEW PREPARATION

Git & GitHub Interview Q&A

12 questions that actually get asked — short answers first, then the detail the interviewer wants.

12 questions | Level: **Junior → Mid** | Format: **Q + model answer**

Q1 What is the difference between Git and GitHub?

Git is a distributed version control system installed on your machine. GitHub is a cloud platform that hosts Git repositories and adds pull requests, issues, Actions and permissions. Git works offline; GitHub needs a network.

Q2 Merge vs rebase — when do you use each?

`merge` keeps history and adds a merge commit; use it for shared branches and PR integration. `rebase` replays commits on a new base for a linear history; use it on your own unpushed feature branch. Never rebase a shared branch.

Q3 Explain `reset --soft` , `--mixed` and `--hard` .

All move the branch pointer. `--soft` keeps changes staged, `--mixed` (default) keeps them unstaged, `--hard` discards working-tree changes. Use `--hard` only on commits you are sure you do not need.

Q4 Reset vs revert — which is safe on a pushed branch?

`revert` is safe: it creates a new commit that undoes an old one, so history is not rewritten. `reset` rewrites history and then needs a force push, which breaks teammates' clones.

Q5 What is the difference between `fetch` and `pull` ?

`fetch` downloads remote refs but does not touch your working tree. `pull` is `fetch` plus `merge` (or `rebase` if configured). Fetch first when you want to inspect changes safely.

Q6 How do you resolve a merge conflict?

Run `git status` to list conflicted files, edit the `<<<<<<<` / `=======` / `>>>>>>>` blocks, remove the markers, test, then `git add` the files and `git commit`. Use `git merge --abort` to bail out.

Q7 What happens when you open a pull request?

GitHub compares your branch with the base branch, shows the diff, and starts checks (CI, review apps). Reviewers comment and approve, required checks must pass, then the PR is merged by merge commit, squash or rebase.

Q8 What are branch protection rules?

Rules that stop direct pushes to important branches. Typical settings: require a PR, 1–2 approvals, passing status checks, up-to-date branch, conversation resolution, and no force pushes or deletions.

Q9 What is `cherry-pick` and when is it useful?

It copies one specific commit onto the current branch. Useful for a hotfix that must go to both `main` and a release branch without merging the whole feature.

Q10 How does `git reflog` help recover lost work?

Reflog records every position of HEAD, including commits removed by reset or rebase. Find the hash, then `git branch rescue <hash>` or `git reset --hard <hash>`.

Q11 What is CI/CD, and where does GitHub Actions fit?

CI builds and tests every change automatically; CD deploys the tested artifact. Actions is GitHub's workflow engine — YAML in `.github/workflows` triggered by push, PR or schedule.

Q12 Lightweight vs annotated tags? Fork vs clone?

Annotated tags store a message, tagger and date — use them for releases; lightweight tags are just pointers. A clone copies a repo to your machine; a fork is your own server-side copy used to propose changes to someone else's repo.

ANSWER LIKE A DEVELOPER

Definition solli mudinjadhukku aprom, **one real example** kudunga: "In my last project we rebased feature branches daily and protected `main` with one approval plus CI." That sentence proves you have used it, not just memorised it.

APPENDIX D · HANDS-ON PRACTICE

Practice Tasks & 7-Day Plan

Paadhi padichadhu mudinjadhula. Ippo keyboard-la practice pannunga — 10 graded tasks, 7 days.

10 tasks | Plan: **7 days** | Level: **Beginner → Advanced**

D.1 Ten Graded Tasks

#	Task	Level	You prove
1	Init a repo, make 3 commits, push to GitHub	Beginner	Local → remote
2	Stage a file, unstage it, discard a change	Beginner	Staging control
3	Create a deliberate conflict on two branches and resolve it	Intermediate	Merge confidence
4	Delete a branch, then recover it with `reflog`	Intermediate	Safe recovery
5	Rebase a feature branch and squash 3 commits into 1	Intermediate	Clean history
6	Open a PR with a template, linked issue and screenshots	Intermediate	Team communication
7	Add a GitHub Actions workflow that runs tests on every PR	Advanced	CI automation
8	Protect `main`: 1 approval, green CI, no force push	Advanced	Repo governance
9	Tag `v1.0.0` and publish release notes	Intermediate	Release flow
10	Cherry-pick a hotfix from `main` to a release branch	Advanced	Production reflexes

D.2 Seven-Day Study Plan

1. **Day 1 — Foundations**
   Ch 1–2. Install, configure, init, stage, commit. Task 1.
2. **Day 2 — History & branches**
   Ch 3–5. Log, diff, branch, merge. Tasks 2 & 3.
3. **Day 3 — Undo & recover**
   Ch 6. Stash, reset, revert, reflog, tags. Tasks 4 & 9.
4. **Day 4 — GitHub collaboration**
   Ch 7. Remote, fork, PR review, issues. Task 6.
5. **Day 5 — Automation**
   Ch 8–9. Actions, checks, Dependabot, secrets. Task 7.
6. **Day 6 — Team simulation**
   Ch 13. Two-person PR flow, protection, release. Tasks 5, 8 & 10.
7. **Day 7 — Polish & interview**
   Ch 10–12, appendices. Profile README, mock Q&A, portfolio repo.

PRACTICE RULES

Always use a **scratch repo** for destructive experiments. Break it on purpose, then fix it. Reading commands gives knowledge; typing them gives skill. Daily 45 minutes > one 6-hour weekend session.

CONCLUSION

You Are Now Git-Ready

13 chapters, one real workflow, and a habit that will follow you through every job.

Neenga ippo `git init` muthal production release varaikkum full path paathuteenga. Commands mattum illa — **why** and **when** also. Ithu dhaan oru developer-ah mattra main difference.

1 Think in commits
One logical change, one commit. Small commits make review easy, revert safe and history readable.

2 Branch small, merge often
Short-lived branches + early PRs = fewer conflicts. Long branches are where merges go to die.

3 Automate the boring part
CI, branch protection and templates protect the team from mistakes — including your own.

Your 30-Day Plan

1. **Week 1 — Fluency**
   Daily commit in a practice repo. Tasks 1–4. No copy-paste — type every command.
2. **Week 2 — History skills**
   Rebase, squash, cherry-pick, reflog. Break a branch and recover it twice.
3. **Week 3 — Collaboration**
   One PR per day with a real description. Review someone else's code politely and specifically.
4. **Week 4 — Portfolio**
   Publish one project with README, CI badge, release tag and a clean commit history. That repo is your proof.

Keep Learning

Resource	Why it helps
Pro Git (free book)	The complete reference for internals and edge cases
GitHub Docs	Current UI, Actions syntax, security and API behaviour
GitHub Skills	Interactive courses inside a real repository
Oh My Git!	A game that makes branch and merge mental models visual
GitHub CLI manual	`gh` commands for PRs, issues and releases without leaving the terminal

ONE LAST THING

Tools change, but the habit stays: **pull before you start, commit small, push daily, review kindly, and never hide a mistake.** Git records everything — that is exactly why it is safe to experiment. Break things in a branch, learn, and ship.

Happy committing!

Dhamodharan · Developer Learning Series